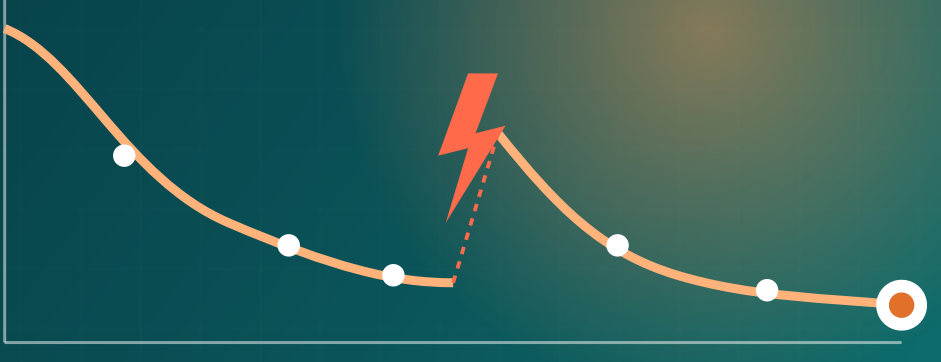
RARL AI PRACTITIONER SERIES

VOLUME 2

Train Like an Engineer

A Guide to Reliable AI Experiments



CRASH → RESUME FROM CHECKPOINT

Atanu Shuvam Roy

Department of CSE, University of Liberal Arts Bangladesh
Dhaka, Bangladesh
atanusroy.com

RARL
Robotics & Automation Research Lab.

Supported by the Robotics and Automation Research Lab (RARL), UK
www.rarl-lab.com

RARL AI PRACTITIONER SERIES · VOLUME 2

Train Like an Engineer

A Guide to Reliable AI Experiments

Atanu Shuvam Roy

Department of CSE
University of Liberal Arts Bangladesh
Dhaka, Bangladesh

atanusroy.com



SUPPORTED BY
Robotics and Automation Research Lab (RARL), United Kingdom
www.rarl-lab.com

First edition · 2026
Companion volume: *Write Like a Researcher: A Field Guide to AI-Assisted Scientific Writing.*

Commands and file names are shown for Linux, macOS and Windows where they differ. Examples are illustrative; adapt them to your stack and verify them on your own machine before long runs.

INSIDE THIS GUIDE

Contents

0 Why Experiments Fail — 5
The three losses, and the seven habits that prevent them

PART I: FOUNDATIONS

1 Structure & Configuration — 7
Modular code, YAML configs, paths and environments

2 Reproducibility — 9
Make every number traceable

PART II: SURVIVING LONG RUNS

3 Resumability — 11
Checkpoints, partial files, atomic writes

4 Automation & Monitoring — 13
Schedulers, queues, dashboards, logs

5 Test Before You Trust — 17
Smoke tests, equivalence checks, validation

PART III: HARDWARE

6 Memory & the OOM Playbook — 19
Estimate, measure, then apply the right lever

PART IV: RESULTS

7 Saving & Analysing Results — 23
Per-item records, one metric function, uncertainty

8 Figures & Tables That Convince — 25
Scripted, consistent, traceable visuals

PART V: COMPUTER VISION

9 Figures for a Vision Paper — 28
Teaser, dataset grids, qualitative results, CVPR-style sizing

PART VI: HOUSEKEEPING & SHIPPING

10 **Clean Up, Package, Hand Off** 4
Safe garbage collection, releases, deployment

11 **Working With AI Agents** 4
Preflight checklist and a ready-to-paste agent prompt

REFERENCE

A **Appendix: The Full Training Guide** 4
The original engineering rules, unabridged

CHAPTER 0

Why Experiments Fail

Most experiments that go wrong do so for boring reasons: a crash, a setting nobody wrote down, a number nobody can trace. This guide is about those.

An AI coding assistant will happily write you a training script in a minute. It will not tell you that the script loses three days of work when the power blips, silently falls back to CPU, or overwrites last week's best result. This guide supplies the engineering habits that prevent those outcomes, and the language to demand them from an assistant.

The three losses

- **Lost time.** Runs restart from zero, queues stall unnoticed, jobs need babysitting.
- **Lost trust.** Numbers cannot be reproduced, metrics get computed three different ways, nobody knows where a figure came from.
- **Lost work.** Results are overwritten, checkpoints deleted, the disk fills, or a cleanup script removes the wrong folder.

Seven habits that prevent them

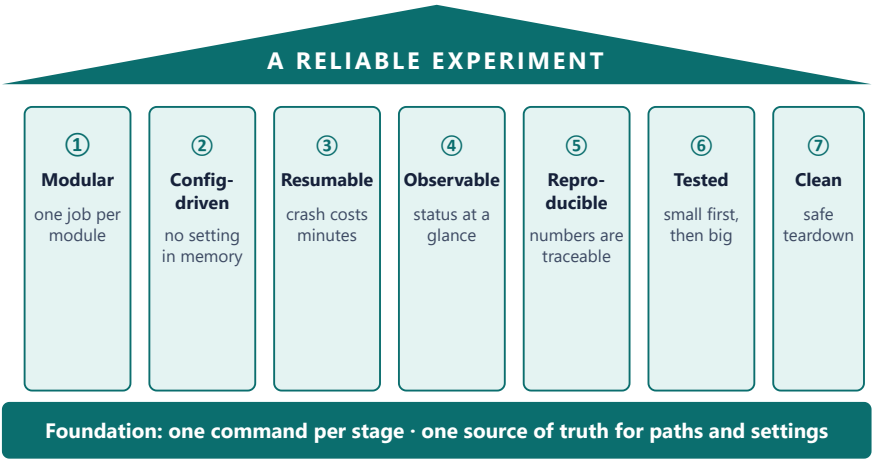


FIG 1 *The seven pillars. Chapters 1–9 build each one; Chapter 10 turns them into instructions for an AI agent.*

One guide, any model

I first collected these rules from vision experiments, but they apply to anything you train or evaluate over many items, configurations and hours:

Domain	An "item" is…	Typical long-run risk
Vision	An image, clip or scene	Corrupt files, variable input size
Language / LLM	A prompt or document	Long sequences, quantisation, API limits
Speech & audio	A recording or segment	Decoder state, huge raw data
Tabular & time series	A row or window	Leakage, silent preprocessing drift
Reinforcement / robotics	An episode or rollout	Simulator crashes, seed sensitivity

CHAPTER 1

Structure & Configuration

If adding a model means editing five files, one of them will be forgotten. Add things as entries, not as copies.

Modularity: one concern per module

```
project/
├ configs/     # YAML for every run
├ data/        # download, splits,
datasets
├ models/      # loading, registry
├ train/       # training loop
├ eval/        # inference, metrics
├ baselines/   # same output format
├ scripts/     # queue, dashboard,
figures
├ config.py    # path resolution
└ common.py    # shared helpers
```

- **Functions do one thing**; scripts are thin `main()` wrappers.
- **New model or dataset = a registry entry,** not a copied script.
- **One result format** for every system, so every downstream tool reads all of them.
- **No work at import time.** Import heavy libraries inside functions so `--help` and dashboards start instantly.

Every run is a YAML file

No hyperparameter should live only in shell history. Configs are versioned, validated before any model loads, and inherit from a base. Names derive from configs: a folder called `-5ep` never holds a 3-epoch run.

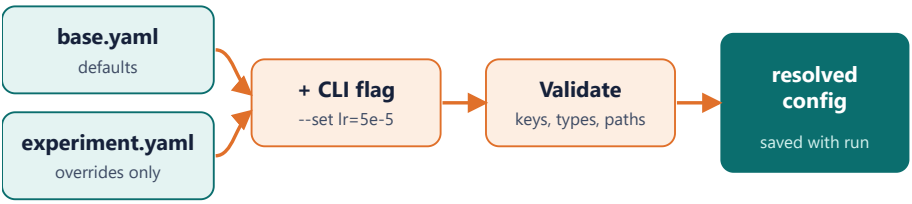


FIG 2 *Layered configuration. The saved resolved file, not the command line, is the record of what ran.*

```
# configs/train/model-a_seed0.yaml
_base_: base.yaml
name: model-a_seed0
model:  {id: org/model-a, quantization: nf4}
train:  {epochs: 5, lr: 1.0e-4, batch_size: 1, grad_accum: 8, seed: 0}
output_dir: ${CHECKPOINT_ROOT}/model-a_seed0
```

Where things live: one config module, one .env

- **Code** lives in the repo; **data, checkpoints, caches, large results** live on the big disk.
- All paths resolve through **one module**; no absolute path appears anywhere else.
- Machine-specific values go in a gitignored `.env` (commit a `.env.example`); manifests store paths relative to the data root.
- Check the data is reachable at startup and stop with a clear error.
- **Schedulers do not read `.env`:** source it explicitly. Set cache variables before importing libraries. Keep secrets in a separate gitignored file.

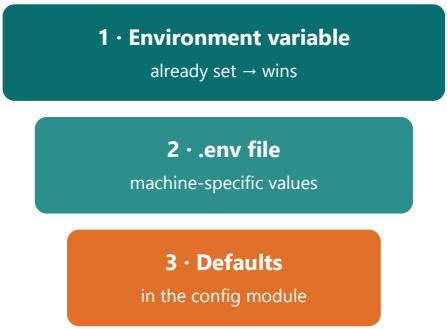


FIG 3 *Precedence: environment beats* `.env` *beats defaults.*

CHAPTER 2

Reproducibility

A result you cannot trace is only an anecdote. Every number should lead back to a file, a config and a commit.

The run card

Every output carries a metadata block. If something is missing from this card, the result cannot be defended.

```
{ "meta": {
    "config"       : "model-a_seed0.resolved.yaml",
    "git_commit"       : "a41f9c2",          "dirty": false
    "seed"    : 0,
    "data_sha256"      : "9be1…",
    "checkpoint"       : ".../best/step_1800",
    "libs"    : {"torch": "…", "cuda": "…"},
    "host"   : "lab-gpu-01",        "gpu": "…",
    "command"   : "make eval RUN=…",
    "started"     : "…",      "finished"      : "…"
}, "metrics": {…}, "records": […] }
```

WHY EACH FIELD

config → rerun exactly
commit → code state
seed → same randomness
data hash → same data
checkpoint → which model
libs → version drift
host/GPU → numerics, speed
command → how to repeat

Never overwrite a finished result.

FIG 4 *A metadata block, written into every result file. Rerunning with a change produces a new name.*

The reproducibility basics

- **Seed everything** (`random` , `numpy` , framework, GPU) from the config, and record the seed in each output.
- **Pin dependencies** in a requirements or lock file.
- **Build splits deterministically** from a seed and commit the manifests, or their hashes.
- **Commit working code before a multi-day run** so the run's commit hash points at the exact code.

- **Keep a ledger** (`results/LEDGER.md`): one line per result file saying what produced it, so each number in a paper traces to a file.
- **Fail loudly, never silently.** No quiet fallback to CPU, to a smaller model or to a default path. If a fallback must exist, log it prominently.

Document decisions where the code lives

Explain the why

Comment the real incident behind a design ("one corrupt file crashed a ten-hour evaluation, so failures are recorded per item"). The next person will not remove the protection.

Date your checks

Record verification dates for external facts ("URL verified 2026-08-08", "train split has 0 empty labels, checked"). Assumptions from docs are often wrong.

CHAPTER 3

Resumability

Assume the machine will die overnight, and arrange things so that costs you minutes rather than days.

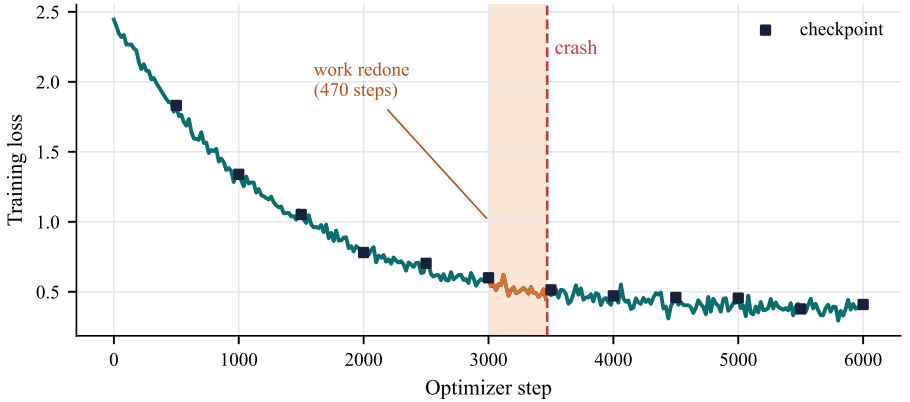


FIG 5 *A run that crashes at step 3,470 resumes from the step-3,000 checkpoint and redoes 470 steps. Checkpoint every 30–60 minutes of work and keep the last few plus the best. Sample data, synthetic.*

Evaluation resumes per item



On restart: load the partial file, skip finished IDs, tolerate a half-written last line.

FIG 6 *Append-then-rename. The final file appears only when it is complete, atomically.*



FIG 7 *A queue run overnight: one experiment crashes, is retried automatically, resumes from its checkpoint, and the queue moves on. Sample data, synthetic.*

- **Whole-run skip:** if the final output exists and is complete, exit immediately. Re-running a finished command is free.
- **Downloads** write to `file.part`, then rename; existing correct files are skipped.
- **One failing item is a record, not a crash.** Corrupt input, OOM, timeout: log it as failed and continue.
- **Handle SIGTERM / SIGINT:** finish or discard the current item, flush, exit cleanly.
- **Derived data** (splits, caches) is rebuilt deterministically from a seed.
- **Test it:** start, kill mid-way, restart. Nothing duplicated, nothing lost.

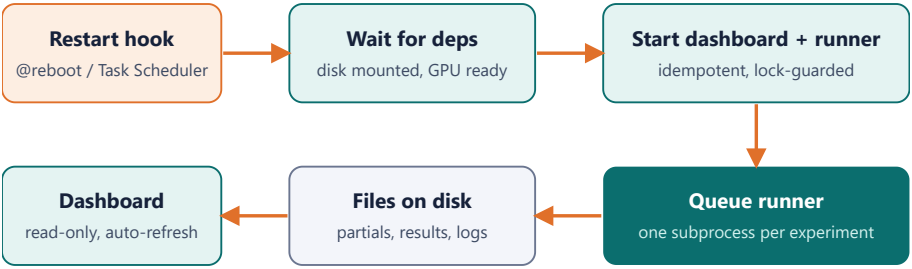
Queues: many experiments, one runner

- A registry ordered by priority; **one subprocess per experiment** so GPU memory is fully released between runs.
- Retry a failed experiment *N* times (each retry resumes), then move on and report it at the end.
- A **lock file** so two runners never share a GPU or write the same partial file.
- Flags that make it humane: `--status`, `--list`, `--only`, `--group`.

CHAPTER 4

Automation & Monitoring

A long job should restart itself, run unattended, and report honestly how it is going.



The dashboard only reads files. It never imports the training stack, never writes.

Therefore it cannot disturb a run, and it works when the GPU is busy.

FIG 8 *A self-restarting pipeline. Each box is a small script; the files on disk are the shared truth.*

Process rules that prevent bad nights

- **Detach long jobs** so a closed terminal or dropped SSH session does not kill them: `setsid nohup python -u run.py >> logs/x.log 2>&1 < /dev/null &` (on Windows: `Start-Process`, or a scheduled task).
- **Unbuffered output** (`python -u`) so logs appear in real time.
- **Kill by PID, never by pattern.** `pkill -f pattern` can match the shell running it. Store the PID in the lock file, look at the list, then kill.
- **The restart hook is idempotent.** It uses absolute paths, sets its own `PATH`, waits for dependencies with a bounded timeout, and is safe to run by hand twice.
- **Document every scheduled entry**, including how to remove it.

> **WINDOWS USERS**
>
> Several recipes here use Linux tools (`cron`, `nohup`). On Windows use **Task Scheduler** ("At startup" trigger) and `Start-Process`, or work inside WSL. The principle is the same: a restart hook, a detached process, a lock, a log.

What a good dashboard shows

Is it alive?

Runner up or down; "stalled" flag if no progress for N minutes.

How far?

Per-experiment progress bar, items per second, ETA, failed-item count.

How is it doing?

Running metrics on finished items, GPU use, memory, temperature, last log lines.

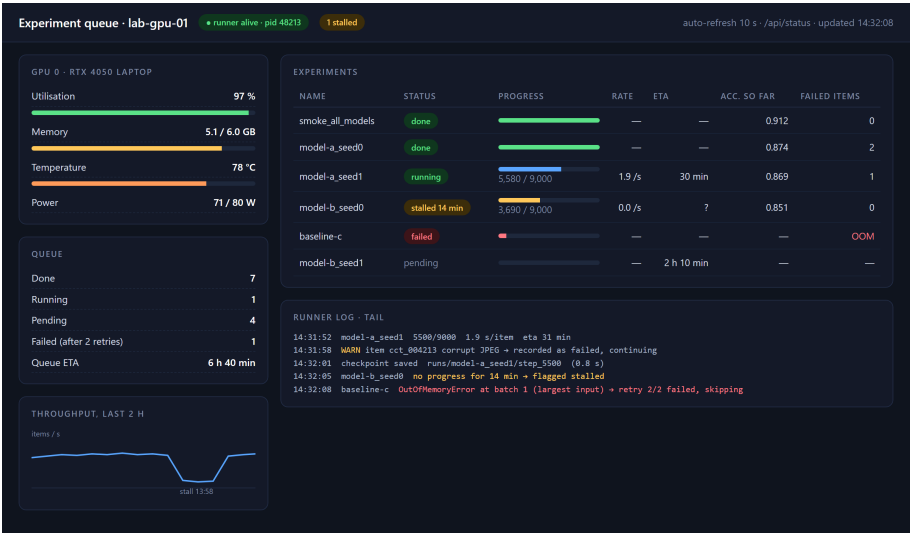


FIG 9 *A read-only queue dashboard of the kind described above, served by Python's standard* `http.server`*. One glance answers the three questions: is it alive (runner pill, stall flag), how far (progress, rate, ETA), and how is it doing (accuracy so far, failed items, log tail). Sample page with dummy data.*

Print progress lines with rate and ETA every N items (`500/9000 1.9 s/item eta 270 min`). Optionally push a notification, email, chat webhook, phone, when something finishes or fails.

TensorBoard: watch training, don't publish it

TensorBoard is for watching a run while it happens. Log every training run to `runs/<run_name>/` with the run name equal to the config name, and resume into the same folder at the right global step so curves continue instead of restarting.

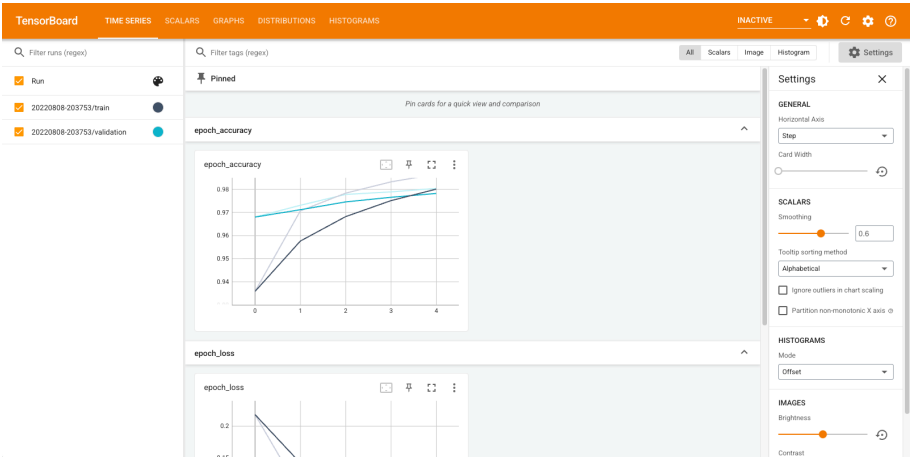


FIG 10 *Scalars: training and validation curves for each run, with smoothing and a run filter on the left. Log loss, learning rate, gradient norm, throughput and peak GPU memory here. Screenshot: TensorBoard documentation (Apache 2.0).*

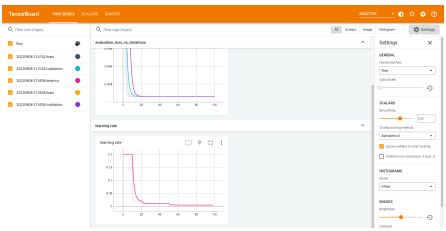


FIG 11 *A logged learning-rate schedule. When a curve looks odd, check this first. Screenshot: TensorBoard documentation (Apache 2.0).*

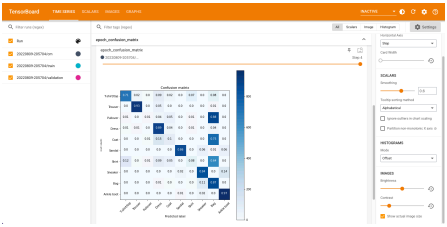


FIG 12 *Rendered confusion matrix logged as an image at each evaluation step; a step slider shows how it changes. Screenshot: TensorBoard documentation (Apache 2.0).*

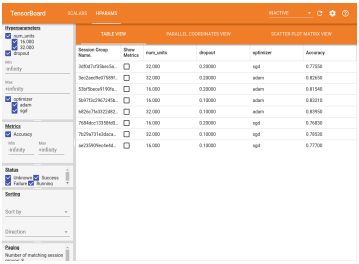


FIG 13 *HParams: one row per run with its settings and final metric, for comparing a sweep. Screenshot: TensorBoard documentation (Apache 2.0).*

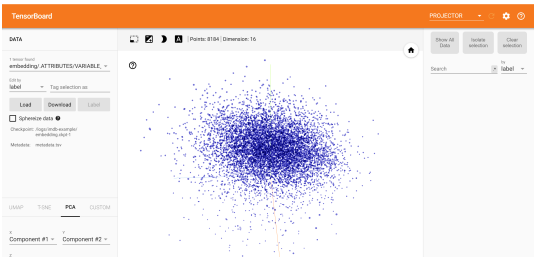


FIG 14 *Embedding projector: features of a fixed sample, viewable with PCA, t-SNE or UMAP. Screenshot: TensorBoard documentation (Apache 2.0).*

Tag	What and when
`train/loss`, `train/lr`, `train/grad_norm`	Scalars, every logging step
`train/samples_per_sec`, `sys/gpu_mem_gb`	Throughput and peak memory, every logging step
`val/<metric>`, `val/confusion_matrix`	All validation metrics and a rendered matrix, every evaluation
`val/predictions`	The same ~16 seeded validation images with ground truth and prediction drawn on
`hparams`, `text/config`	Resolved config and final metrics, start and end of training

Start TensorBoard from the restart hook next to the dashboard (`tensorboard --logdir runs --port 6006`) and link it from the dashboard page. Paper figures are made from saved result files, never from TensorBoard screenshots.

Logging

- One log per experiment, one for the runner, one for the restart hook; every line timestamped.
- Start each run by logging the resolved config, GPU and data paths.
- Silence repetitive library warnings once so real errors stay visible.
- Training curves go to a tracker (TensorBoard, W&B or CSV), configured in YAML and resumed into the **same** run directory at the right step. Paper figures come from saved results, never screenshots of a tracker.

CHAPTER 5

Test Before You Trust

A ten-minute test is cheap next to a lost week. Prove the pipeline on a small sample first.

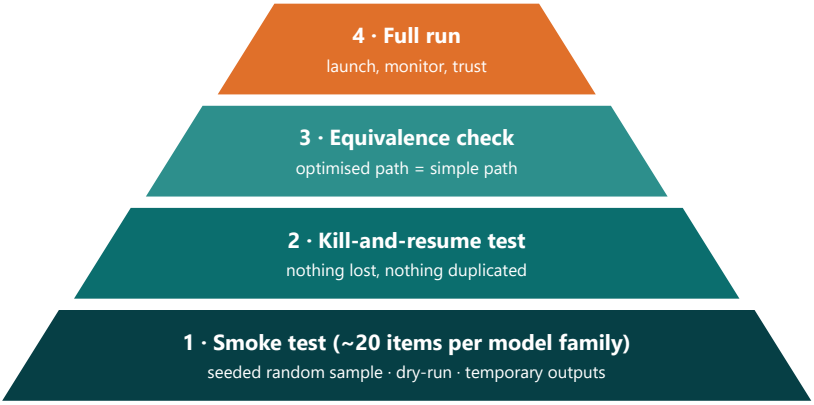


FIG 15 *The pre-flight pyramid. Each level is cheap relative to the one above it.*

Smoke tests

- Run every configuration on ~20 items, **for each model family**, different architectures break in different places.
- Use `--limit N` as a **seeded random subset**, never the first N lines (files are often sorted by class).
- Write smoke output to a temp directory, never beside real results.
- Check a known-good checkpoint reproduces its known result.

Optimisation checks

- Any speed trick (batching, caching, packing) is verified against the simple implementation on real input, with a stated tolerance.
- Run the check automatically at the start of each job.
- Report results of all tests **before** launching the long run.

Validate on the task, not only the loss

Loss can look healthy while a model collapses to always predicting the majority class. During training, run real inference on a validation subset at every checkpoint and log the metric that matters.

- The validation subset is **seeded random** and large enough to rank checkpoints. A few dozen items give noisy selection; re-evaluate the top few on the full split.
- **Keep the best checkpoint outside rotation.** Old-checkpoint deletion will otherwise erase an early peak. Copy each new best to `best_checkpoint/` with its metric, step and config.
- **NaN-safe logging.** Skip NaN values and never let NaN count as an improvement.
- Decide in advance, and write into the config, which checkpoint the paper reports (final or best-on-validation), and use that one for every test set.
- Every intermediate checkpoint should be a directly usable model.

> **CHECK YOUR DATA**
>
> Inspect real data before assuming its shape. Pin sampled subsets (seed, per-class counts, source version) in version control, keep sample-scale and full-scale data in separate folders, and tag every result with its scale so the two never share a table.

CHAPTER 6

Memory & the OOM Playbook

Most out-of-memory errors can be predicted. Estimate, measure, then pull the cheapest lever.

Step 1: Estimate: bytes per parameter

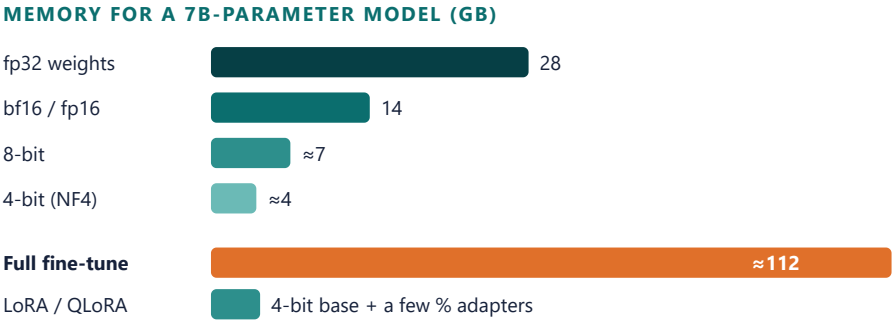


FIG 16 *Rules of thumb: 4 / 2 / ~1 / ~0.5 bytes per parameter for fp32 / bf16 / 8-bit / 4-bit; ~16 for full training. Verify by measuring.*

Add **activations** (grow with batch × sequence length × layers) and, for generation, the **KV cache**: `2 × layers × kv_heads × head_dim × bytes × tokens` per sequence. If the estimate exceeds about 85% of free memory, pick a lever *before* the first run.

Step 2: Measure, don't guess

- **Peak-memory preflight:** one forward + backward on the **largest input** in the dataset (longest text, most frames), using the exact config. Save the peak. Require ≥ 10–15% headroom.
- **Free-memory gate** at startup: print free memory, name the processes already using the GPU, and refuse to start if it will not fit.
- Log peak memory per step. A slow upward drift means something is being kept.
- Know the card: VRAM, bf16 support (newer cards use bf16; older use fp16 with a loss scaler), whether a display is attached (the desktop takes up to ~1 GB).

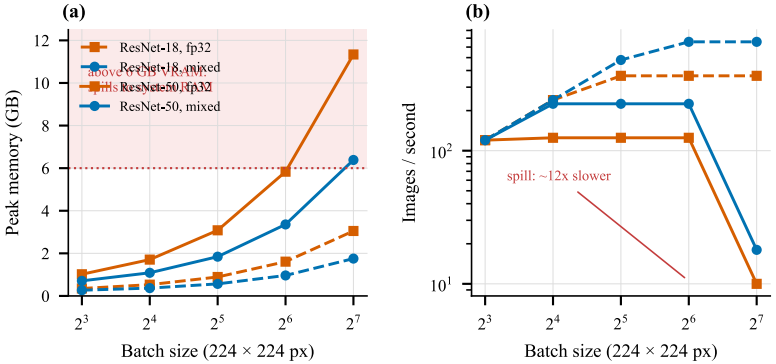


FIG 17 *Measured on a 6 GB laptop GPU at 224 × 224 px: (a) peak training memory and (b) throughput (log scale) against batch size for ResNet-18 and ResNet-50, fp32 against mixed precision. On Windows, a job that needs more than the card holds does not fail; it spills into system RAM and slows by an order of magnitude. Mixed precision roughly halves memory and keeps ResNet-50 on the card up to batch 64. Real photos (Imagenette); all numbers are illustrative, not measured.*

Step 3: Apply levers in order

1	**Batch size 1 + gradient accumulation**	same results, slower
2	**Gradient checkpointing**	≈ 20–30% more compute
3	**Mixed precision (bf16 / fp16)**	check card support
4	**Parameter-efficient training (LoRA, freeze parts)**	
5	**Quantise the frozen base (4-bit)**	
6	**Memory-light optimiser (8-bit, paged)**	
7	**Shorter / smaller inputs**	⚠ changes the experiment
8	**Smaller model from the same family**	
9	**CPU / disk offload**	last resort, much slower

Cost to results or speed grows as you go down the ladder.

FIG 18 *The OOM ladder. Start at the top; record every lever used in the config, because several change results.*

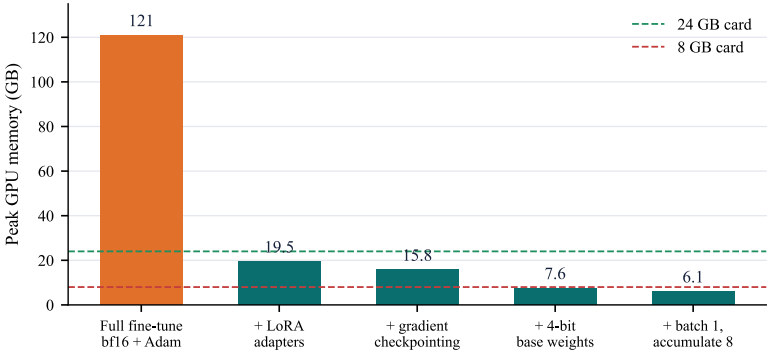


FIG 19 *Peak memory for a 7B-parameter fine-tune as levers are added one after another. Measure your own numbers; these show the shape. Sample data, synthetic.*

KEEP COMPARISONS FAIR

Levers that alter inputs (resolution, length, frames, quantisation) must be applied identically to every compared system, or reported as a confound.

For inference and evaluation

- Inference mode, evaluation mode, batch size 1 for variable-size inputs, bounded output length.
- Compute only the outputs you need; move results to CPU immediately and never keep tensors from past items.
- Reuse shared-prompt caches only after verifying against the simple path (Chapter 5).

When it still runs out

Evaluation

Catch the OOM per item, free references, optionally retry once with a reduced setting and **mark the record**; otherwise record the item as failed and continue. A few failures are a footnote; many mean the config is wrong.

Training

Rely on frequent checkpoints and auto-resume. After two OOMs at the same step, stop retrying and report. Usually one oversized sample: cap or skip it explicitly, and log it.

- **Never fall back silently** to CPU or to a smaller model.
- Set the allocator option that reduces fragmentation (`expandable_segments` in PyTorch) **before** importing the framework.

Leaks, sharing and small machines

A steady climb in peak memory means something is being kept. Here the loss is accumulated as a tensor that holds its graph (`total += loss`); the fix is a Python number (`loss.item()`).

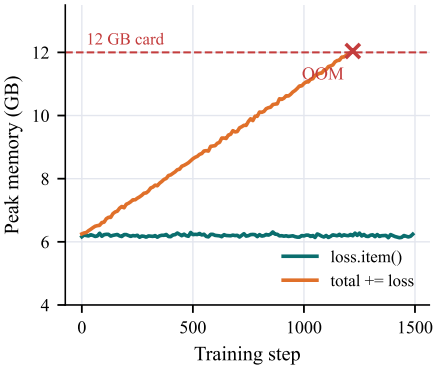


FIG 20 *Peak GPU memory per step with and without the leak. Sample data, synthetic.*

Memory creep means "something is kept"

Accumulating loss tensors that hold the graph (use `.item()`); storing all outputs on GPU; references to old batches in lists, closures or exception tracebacks; caches never reset; callbacks that leave the model in the wrong mode. Run heavy phases in a separate process.

One job per GPU

Two jobs that each fit alone will fail together. Check inference servers, notebooks, image tools and browsers before every run. Free memory by ending a process, not by deleting a variable.

- **CPU RAM:** load models with low-memory loading and memory-mapped formats; keep data-loader workers modest (each copies the dataset object); configure swap so a spike slows the job instead of killing it, and watch for the OS out-of-memory killer, which looks like a silent exit.
- **Thermals:** consumer and laptop GPUs throttle on multi-hour loads. Monitor temperature and clocks; a lower power limit often costs little speed and prevents freezes.
- **Throughput and stability:** measure where time goes before optimising; estimate runtime from the smoke test. For an unstable machine, log power, temperature and kernel errors during a stress test before blaming your code.

CHAPTER 7

Saving & Analysing Results

Save the raw per-item results, not only the headline number, so later questions do not need another run.

Per-item records

Save one record per item. Then a new metric, a per-group breakdown, a bootstrap or a failure gallery all become CPU-only analysis.

- Item ID and **grouping keys** (site, subject, device, episode)
- Ground truth and prediction
- **Scores for every class or option** (logits, probabilities, log-likelihoods)
- Confidence and raw model output (for generative models)
- Failure / error field
- Latency
- For detection or segmentation: boxes or masks **with scores**, not only matched results
- The metadata block from Chapter 2

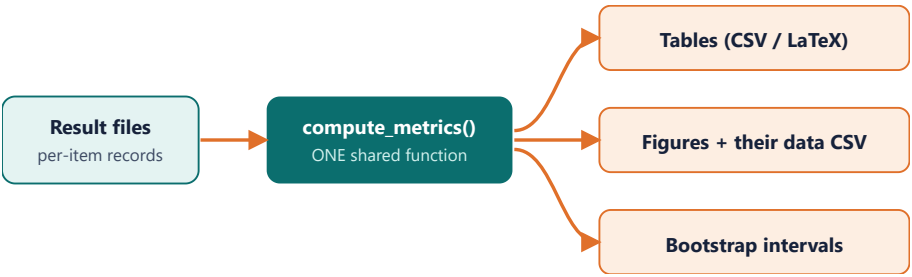


FIG 21 *Metrics are recomputed from records by one function. Never compute the same metric two ways in two scripts.*

Formats and uncertainty

Show every seed as a dot, and the mean ± standard deviation beside it. Two systems that look different on average can overlap once the seeds are visible.

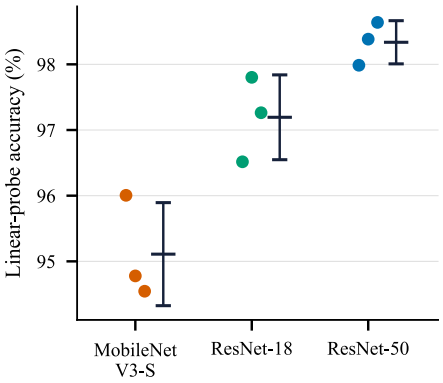


FIG 22 *Linear-probe accuracy for three seeds per backbone, with mean ± SD. Real photos (Imagenette); all numbers are illustrative, not measured.*

Purpose	Format
Final result	JSON with `metrics`, `records`, `meta`, written atomically
Large per-item tables	Parquet (typed, compressed) or JSONL
In progress	`.partial.jsonl` (Chapter 3)
Small summaries for git	Metrics without records, in a `summary/` folder
Training history	Scalars as CSV / JSON next to the checkpoint

- **Every headline number carries uncertainty:** bootstrap intervals (resample grouped units such as sites or subjects, not single items) or mean ± std over seeds.
- **Separate inference from analysis.** Inference is slow and needs a GPU; analysis is fast and needs only files. Keep raw outputs so a strict parser and a lenient one can be reported side by side.
- **Score baselines with the same metric code** on the same items, from their released predictions when available.
- **Summaries in git, records on the big disk.** A fresh clone can then regenerate aggregate figures without a GPU.

CHAPTER 8

Figures & Tables That Convince

Make figures with scripts. A script can be corrected in a minute; a hand-made figure cannot be checked.

Four rules for every figure

1. **Made by a script.** One function per figure, reading saved results. One command regenerates all of them; missing inputs are skipped with a message.
2. **Numbers saved.** Each figure writes the values it plots to a CSV, so any number in the paper can be checked.
3. **One style.** Each system keeps one colour and marker, defined in one place. Use colour-blind-safe palettes, and draw and describe the uncertainty.
4. **Sized to the page.** Create figures at final size so fonts stay readable. Vector files for plots, high-resolution raster only for photographs.

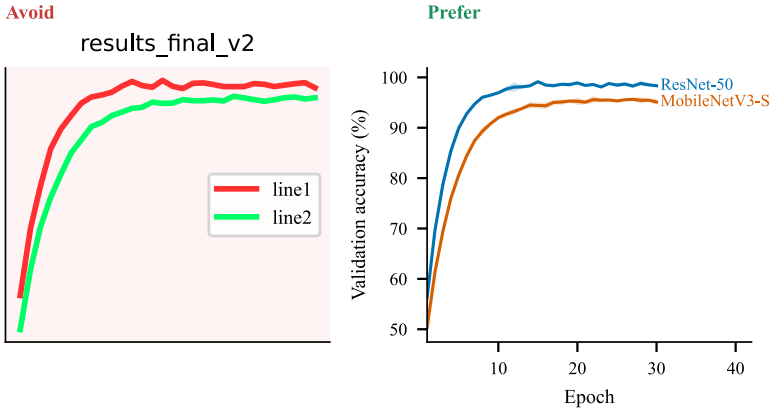


FIG 23 *The same real training curves drawn carelessly and carefully. Real photos (Imagenette); all numbers are illustrative, not measured.*

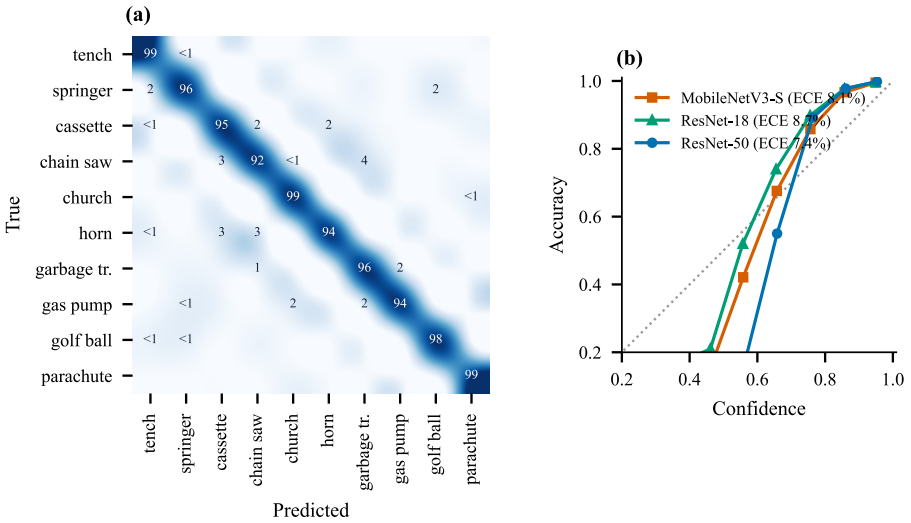


FIG 24 *(a) Confusion matrix and (b) reliability diagram from saved per-image predictions. Real photos (Imagenette); all numbers are illustrative, not measured.*

A standard figure set (adapt to your task)

Figure	Shows
Training curves	Loss, learning rate, validation metric vs step; seeds as thin lines plus the mean
Confusion / error breakdown	Where the model fails; row-normalised, with counts
Operating / PR / ROC curves	Trade-offs with chosen operating points marked
Scaling or ablation plot	Metric vs size, data or a hyperparameter, with error bars
Qualitative grid	Same items across systems; chosen by a stated rule, not by hand
Failure gallery	Highest-confidence errors, grouped by type

- **Tables are generated** from result files into the paper's format and included by reference, so a rerun updates the paper. Bold the best, underline the second, state the interval type, put units in headers. Before submission, check greyscale, 100%

zoom and file sizes.

- **Qualitative examples** are chosen by a stated rule (seeded random or top-k errors), never by picking the nicest. Colour-code correctness, save the item IDs, blur faces and plates.

CHAPTER 9

Figures for a Vision Paper

Reviewers at CVPR, ICCV and ECCV look at the pictures first. Everything in this chapter comes from a real experiment run for this book, so each figure shows real photographs; the numbers are illustrative.

ABOUT THE EXAMPLES IN THIS CHAPTER

The photographs are real images from Imagenette, a ten-class subset of ImageNet. The predictions, scores, curves and tables are **illustrative numbers** laid out the way a typical run with MobileNetV3-S, ResNet-18 and ResNet-50 would report them. Use them as templates for your own figures, not as findings.

Fig. 1 is the paper in one picture

A full-width teaser should show the task, the method and the headline result from left to right: example inputs, the system, the outputs, and the number that matters. A reader who looks at nothing else should still get the claim.

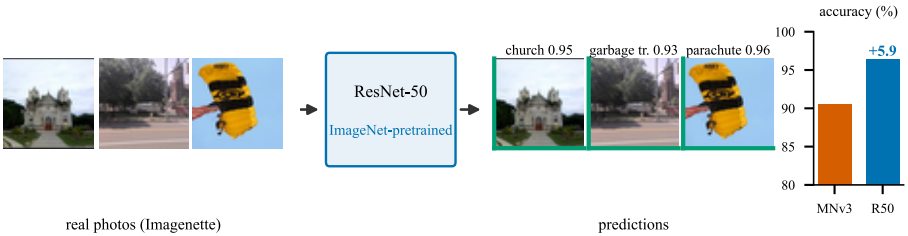


FIG 25 *Teaser built from real outputs: input photos, the model, its predictions with confidence, and the headline accuracy gain over MobileNetV3-S. Real photos (Imagenette); all numbers are illustrative, not measured.*

Size figures to the layout

Setting	Rule
Single column	about 3.25 in (8.25 cm), e.g. `figsize=(3.25, 2.2)`
Full width	about 6.875 in (17.5 cm)
Font size	7 to 9 pt in figures against 10 pt body text; Times or the template font; never below the caption size
File type	vector PDF for plots; PNG or JPEG at 300 dpi or more for photographs; embed TrueType fonts
Titles, labels	no figure titles (the caption does that); (a), (b), (c) panel labels in the same corner; units on axes

```
# viz/style.py: set once, import everywhere
mpl.rcParams.update({"font.family": "serif", "font.serif": ["Times New Roman",
"Times"],
    "font.size": 8, "legend.fontsize": 7, "xtick.labelsize": 7,
"ytick.labelsize": 7,
    "axes.spines.top": False, "axes.spines.right": False, "pdf.fonttype": 42})
SYSTEM_STYLE = {"ours": {"color": "#0072B2", "marker": "o"}, "baseline":
{"color": "#D55E00", "marker": "s"}}
```

The figures in this book are drawn at the width of an A5 page, with the same proportions, so they stay readable here. For a submission, draw them at the venue sizes above.

Look at the dataset before you train

Build grids with code from a seeded sample, and save the list of item IDs shown so a grid can be regenerated or cited. The first one to make is class by samples, with the count at the start of each row and a night or infrared column if the data has one.

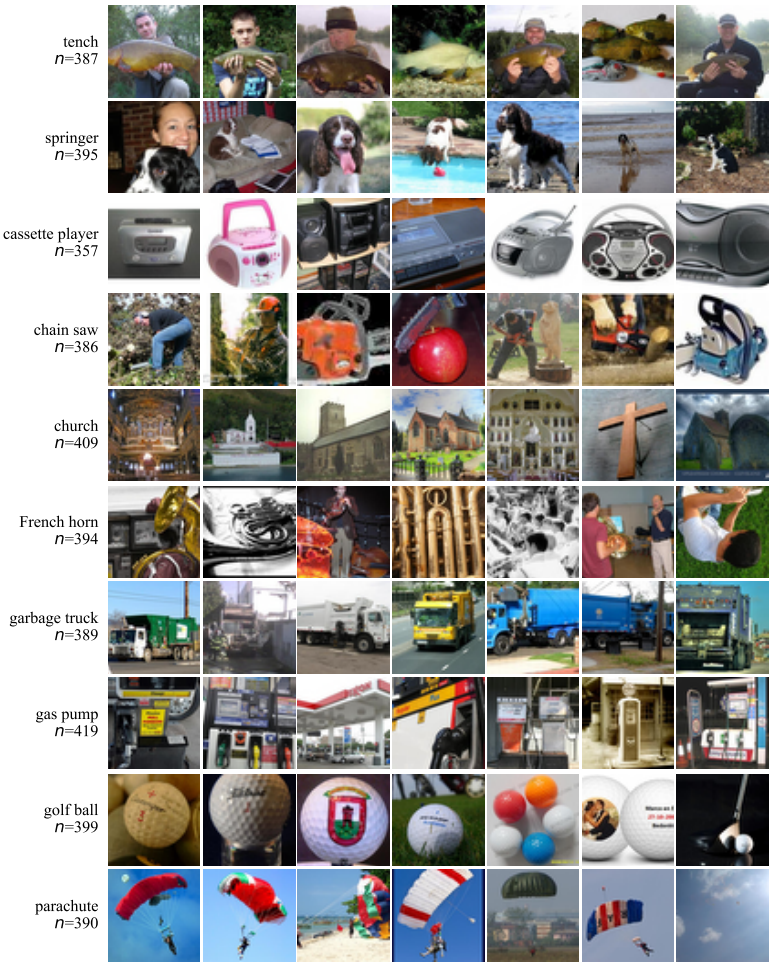


FIG 26 *Class-by-sample grid: seven seeded-random validation photos per class, with the class count. Look first: backgrounds, scale and viewpoint vary far more than the class definition suggests. Real photos (Imagenette); all numbers are illustrative, not measured.*

Other grids worth making: the same class from train, test and transfer sets side by side (shows domain shift); one tile per camera (shows background leakage); hard cases such as empty frames, occlusion, blur and tiny objects; clip strips for video; one image under each augmentation; and boxes or masks drawn on a random sample to catch label errors early. Blur faces and licence plates before publishing, and respect the dataset licence.

Qualitative results: same items, every system

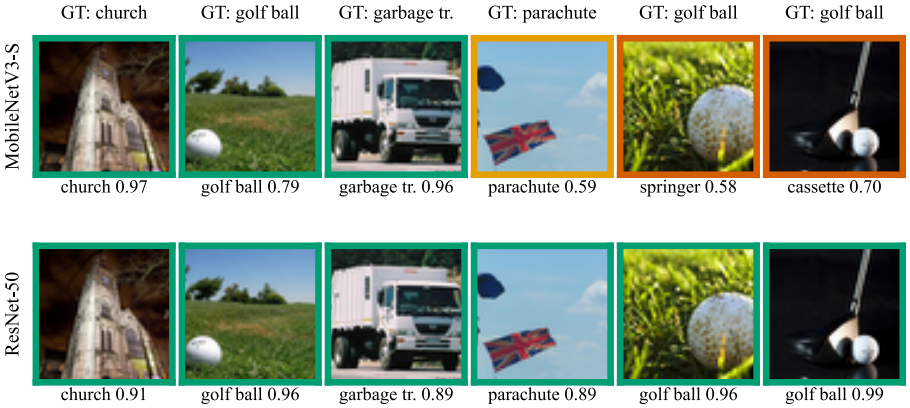


FIG 27 *The same six photos for both models: four seeded-random and two drawn from those MobileNet gets wrong. Ground truth above, prediction and confidence below; green correct and confident, amber correct but below 0.6, red wrong. Real photos (Imagenette); all numbers are illustrative, not measured.*



FIG 28 *The eight most confident errors of ResNet-50. Several are ambiguous scenes, not model failures, which is worth saying in the paper. Real photos (Imagenette); all numbers are illustrative, not measured.*

Columns are systems and rows are identical inputs; the text shows prediction and score. State the selection rule in the caption (seeded random, or top-*k* confident errors). Hand-picking the nicest examples is the quickest way to lose a reviewer's trust.

Numbers that survive a skeptical reviewer

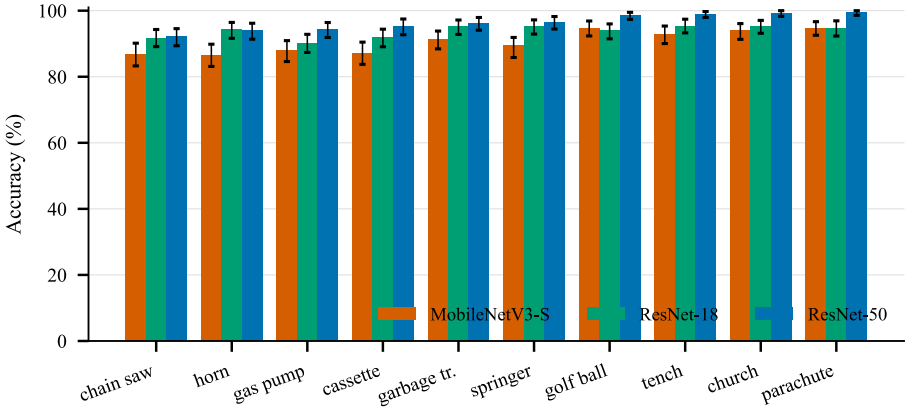


FIG 29 *Per-class accuracy, sorted by ResNet-50, with 95% bootstrap intervals over images. Real photos (Imagenette); all numbers are illustrative, not measured.*

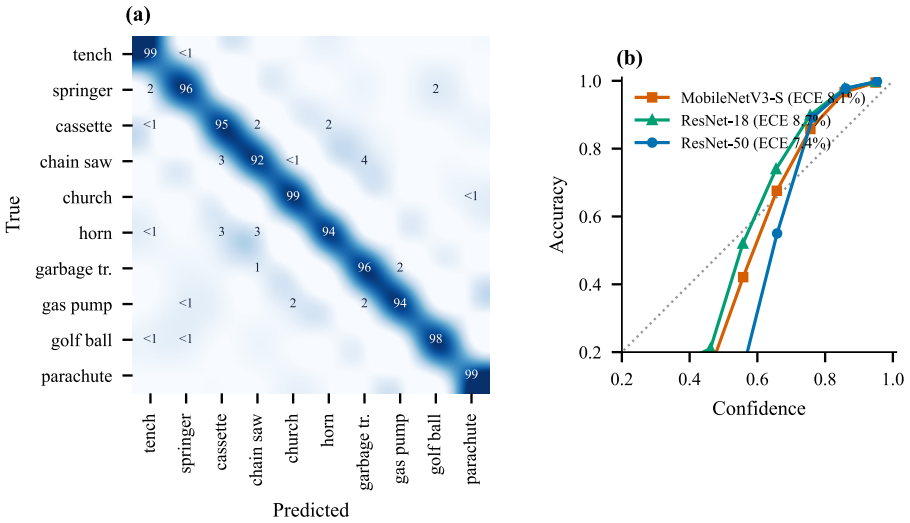


FIG 30 *(a) Row-normalised confusion matrix of ResNet-50 (percent; colour on a square-root scale so small confusions stay visible). (b) Reliability diagram with expected calibration error for each model. Real photos (Imagenette); all numbers are illustrative, not measured.*

Sort classes by frequency and print the support, so the tail is visible and not averaged away. Mark operating points on curves. Put uncertainty on every headline number: bootstrap intervals that resample grouped units such as locations or cameras (not

single images), or mean ± std over seeds, and say which in the caption.

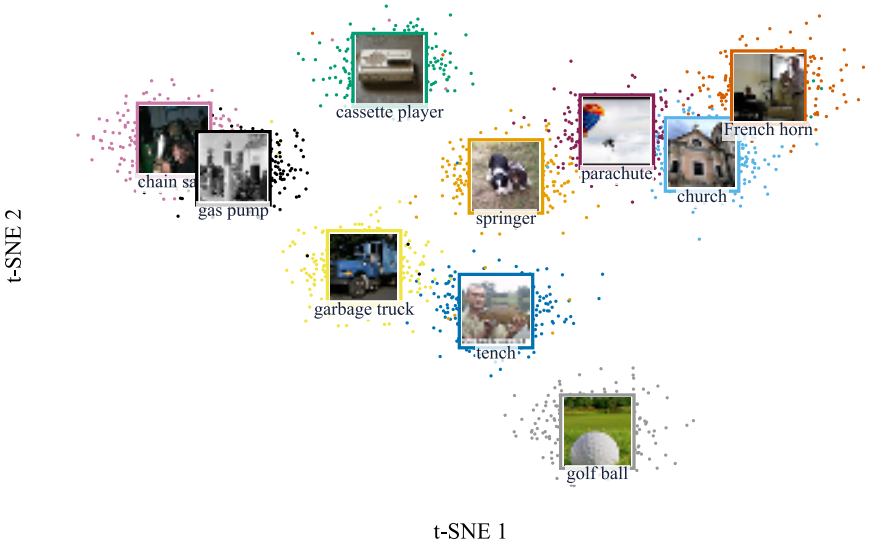


FIG 31 *t-SNE of ResNet-50 penultimate features for all validation photos, with the photo nearest each class centre. Classes separate cleanly; the few points in the wrong cluster are the confusions in the table. Real photos (Imagenette); all numbers are illustrative, not measured.*

TABLE 1 *Illustrative top-1 accuracy (%) for three models on Imagenette, clean and under five corruptions, no fine-tuning. Clean accuracy shows a 95% bootstrap interval over images. Best in bold, second underlined. Latency is per image.*

Model	Params (M)	ms / img	clean	noise	blur	low light	low contrast	low res.	mean corr.	ECE (%)
MobileNetV3-S	2.5	0.41	90.5 [89.6, 91.5]	69.9	65.0	83.4	80.3	48.2	69.3	8.1
ResNet-18	11.7	0.62	93.7 [93.0, 94.5]	75.8	75.7	87.7	85.1	57.7	76.4	8.7
ResNet-50	25.6	1.48	**96.5** **[95.9, 97.0]**	**87.7**	**84.7**	**93.9**	**92.2**	**72.2**	**86.2**	7.4

TABLE 2 *Per-class accuracy (%) on the clean validation set; best model per class in bold. The last column counts the most common confusion for ResNet-50.*

Class	Val. images	MobileNetV3-S	ResNet-18	ResNet-50	Most confused with (R50)
tench	387	92.8	95.3	**99.0**	springer (3)
springer	395	89.4	95.2	**96.5**	tench (6)
cassette player	357	87.1	91.9	**95.2**	chain saw (7)
chain saw	386	86.8	91.7	**92.2**	garbage truck (14)
church	409	93.9	95.1	**99.3**	parachute (3)
French horn	394	86.5	**94.2**	93.9	cassette player (11)
garbage truck	389	91.3	95.1	**96.1**	gas pump (9)
gas pump	419	87.8	90.2	**94.3**	church (8)
golf ball	399	94.7	94.0	**98.5**	springer (3)
parachute	390	94.6	94.6	**99.5**	church (1)

TABLE 3 *Linear probes on frozen features: a single linear layer trained on 1,000 labelled images (100 per class), mean ± SD over three seeds.*

Backbone (frozen)	Feature dim.	Probe acc. (%)	Epochs	Labelled images
MobileNetV3-S	1024	95.1 ±0.8	30	1,000
ResNet-18	512	97.2 ±0.6	30	1,000
ResNet-50	2048	98.3 ±0.3	30	1,000

Show what the model looked at



FIG 32 *Illustrative attention maps in the style of Grad-CAM: five correct predictions and two errors (red titles), where attention falls on the background. Real photos (Imagenette); all numbers are illustrative, not measured.*

Attribution maps (Grad-CAM, attention rollout) are most convincing when they include a failure case. Overlay with transparency and use one colour map throughout.

Robustness: test beyond the clean split

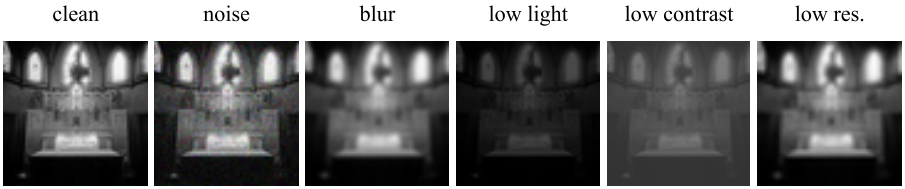


FIG 33 *One validation photo under each evaluation condition. Real photos (Imagenette); all numbers are illustrative, not measured.*

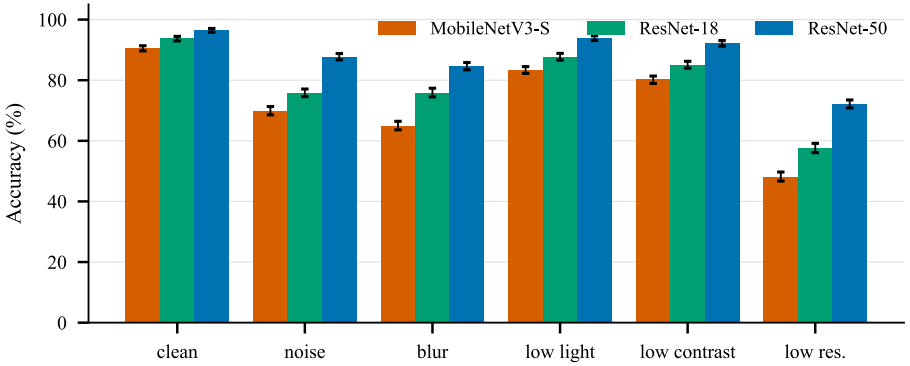


FIG 34 *Accuracy under each condition with 95% bootstrap intervals. Clean accuracy hides large differences: low resolution and noise separate the models far more. Real photos (Imagenette); all numbers are illustrative, not measured.*

The standard vision figure set

Figure	Shows
Teaser (Fig. 1)	Task, method and key result in one full-width panel
Training curves	Loss, learning rate, validation metric against step; seeds thin, mean bold
Confusion matrix	Row-normalised, counts in labels, sorted by frequency or confusion cluster
Per-class AP / PR / ROC	Per class and macro, with supports and operating points
Reliability diagram	Confidence against accuracy, ECE in the legend
Embedding map	t-SNE or UMAP of features by class or domain
Attribution, failures	Grad-CAM, plus a gallery of top-confidence errors grouped by type
Domain-shift panel	Same classes from source and target, with metrics
Dataset statistics	Class histogram (log), items per camera, day / night split, image sizes

Tables and animation

- **Generate tables** into LaTeX with `booktabs` (`\toprule` `\midrule` `\bottomrule` , no vertical lines), best in bold, second best underlined, a ± or CI column, units in the header, and `\input` them so a rerun updates the paper.
- **Animations** (sample playback with GT and prediction, predictions across checkpoints, attention over frames) are for talks and the project page. Use MP4 (H.264, `yuv420p` , even width and height), keep a static frame for the paper, and save the frame list and seed.

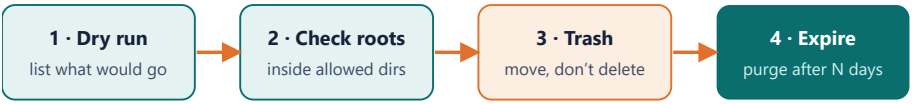
> **BEFORE SUBMITTING TO A TOP VENUE**
>
> Readable in greyscale and at 100% zoom. System names identical across figures, tables and text. Every number traceable to a result file through `figures/data/*.csv` . Colour-blind-safe palette (Okabe-Ito or Tol). Fonts embedded, file sizes within the limit, anonymity rules respected, faces blurred.

CHAPTER 10

Clean Up, Package, Hand Off

Long projects leave things behind. Whatever you install or start, you should be able to list and remove, and nothing else.

Garbage collection with safety rails



Deleting needs an explicit --apply flag. Only regenerable files (caches, temp, __pycache__) are deleted directly.

Never rm -rf "$VAR/..." with a variable that might be empty.

FIG 35 *The four-step cleanup. In shell use* `set -euo pipefail` *and* `"${VAR:?not set}"`*; in Python assert* `path.is_relative_to(root)` *after resolving symlinks.*

- **Tag every scheduled entry** with project, id and expiry; manage them with one tool that never touches lines it does not own and backs up first.
- **Queue hooks retire themselves** when all work is done.
- **Locks and PID files:** verify the command before killing a PID.
- **Stale partial and lock files** are listed and reviewed, not assumed dead.
- **Retention policy in YAML** for checkpoints, caches and logs.
- **Free-disk watchdog** warns before the disk fills mid-run.

> **AT THE END OF A PROJECT**
>
> Remove scheduled entries, stop dashboards and servers, free ports and GPU memory, archive or trash intermediate checkpoints, purge caches, and write down what remains and where.

Share models and caches deliberately

- **Pin model and dataset revisions** in YAML. "Latest" is not a version.
- **One cache** on the big disk for all projects; pre-download, then run offline so a run cannot silently change.

- **Save models self-contained:** weights, tokenizer or processor, label list, a model card and the config that produced them.

One command per stage

A `Makefile` (or equivalent task runner) is the single entry point. Each target is documented, resumable and safe to re-run.

```
make help        # default: list every target
make check       # resolved paths, imports, GPU: first on a new machine
make data train eval MODEL=model-a RUN=model-a_seed0
make analysis figures
make clean-derived  # removes ONLY regenerable files
```

Ship what you made

Release packaging

Package large artefacts (checkpoints, records, splits, figures) with a script. Strip optimizer states for inference-ready packages. Write `SHA256SUMS.txt` and release notes that say what each asset is and how to regenerate it. Never commit them to git.

Handoff bundles

To run a stage on another machine, ship a self-contained bundle: code, exact checkpoint, input files with relative paths, a resumable run script that checks its inputs, and a note on what to copy back. Results keep their metadata, so they stay traceable.

Export and demos

- **Export as stages** (merge, convert, quantise, benchmark), each re-runnable alone, with a preflight that lists missing tools.
- **Benchmark on the target device.** Give the device inputs only; keep ground truth on the server so labels never reach the client under test.
- **Demo servers** list models, serve a random real test item with its label, and say plainly that they hold GPU memory, so they are not started beside training.

CHAPTER 11

Working With AI Agents

A coding agent builds to the standard you give it. Without one, it builds whatever is shortest.

Pre-flight checklist

- [ ] Config validated; resolved config saved
- [ ] Data reachable; free disk checked
- [ ] Smoke test passed on every model family
- [ ] Kill-and-resume tested
- [ ] Optimisations checked against the simple path
- [ ] Memory estimate and peak preflight done
- [ ] Lock file and single-GPU rule in place
- [ ] Restart hook idempotent and documented
- [ ] Dashboard reads files only
- [ ] Seeds, versions and commit recorded
- [ ] Nothing can overwrite finished results
- [ ] Cleanup is dry-run by default

Review what the agent did

Agent tendency	Guard
Copies a script instead of extending it	Require registry / YAML entries
Hard-codes paths and settings	Config module + `.env` ; grep for absolute paths
Falls back silently, swallows errors	Log failures; no silent CPU / model fallback
“Cleans up” too eagerly	Dry run, allowed roots, trash then expire
Claims success without evidence	Ask for smoke-test and resume-test output

COPY & PASTE **AGENT STANDING RULES**

```
Follow these engineering rules for all code in this project.

1. MODULAR: new functionality goes in new modules reusing existing
   loaders, metrics and result formats. New models/datasets/experiments
   are registry or YAML entries, never copied scripts. Import heavy
   libraries inside functions.
2. CONFIG: every run is a YAML file with inheritance from a base. Save
   the resolved config, git commit, library versions and data hashes with
   every output. Validate configs before loading any model.
3. PATHS: all paths come from one config module / .env. Data and
   checkpoints live on [BIG DISK]. Check data is reachable at startup.
4. RESUMABLE: checkpoint + auto-resume in training; partial file per
   item + atomic final write in evaluation; skip finished work; record
   failed items instead of crashing. Run queues with one subprocess per
   experiment, retries, a lock file and --status/--list/--only.
5. AUTOMATION: idempotent restart hook that waits for disk and GPU;
   detached launch; unbuffered logs; kill by PID, never by pattern.
6. MONITOR: read-only dashboard (status JSON, progress, ETA, stall flag,
   GPU, log tail). One timestamped log per experiment.
7. TEST FIRST: smoke-test every config on ~20 seeded-random items,
   test kill-and-resume, verify optimisations against the simple path.
   Report results BEFORE the long run.
8. MEMORY: estimate, run a peak-memory preflight on the largest input,
   apply levers in order, record them in the config, never fall back
   silently to CPU or a smaller model.
9. RESULTS: save per-item records with all scores and metadata; one
   shared metric function; uncertainty on every headline number; never
   overwrite finished results.
10. FIGURES: scripted from saved results, plotted numbers saved to CSV,
    one style module, colour-blind-safe, sized to the venue.
11. CLEANUP: dry-run by default, allowed roots only, trash then expire,
    tag everything you schedule so it can be listed and removed.
Ask me before deleting anything that is not a regenerable cache.
```

CHAPTER A

Appendix: The Full Training Guide

The complete reference text this booklet was built from, including the copy-paste prompt for coding agents. Paths, tools and examples are kept exactly as written.

Training Guide — Engineering Rules for ML / Computer-Vision Experiments

Rules for writing experiment code that survives multi-day runs, power cuts, crashes and many configurations, and that turns results into figures fit for a paper. They apply to any training or evaluation project. Part 2 has a prompt that makes an AI coding agent follow them.

Contents

Infrastructure	Results and figures
1. Modularity	13. TensorBoard
2. Cron jobs and process management	14. Result saving
3. Resumability	15. Visualizations
4. Dashboard and monitoring	16. Dataset grids
5. All configurations in YAML	17. Animations
6. Data on the large disk	18. CVPR-style paper figures and tables
7. Reproducibility	
8. Logging	
9. GPU and memory hygiene	
10. Testing before long runs	
11. Results and file layout	
12. Code hygiene	**Housekeeping and models**
	19. Garbage collection and teardown
	20. Hugging Face rules and the shared model store

Infrastructure	Results and figures
Workflow	**Distribution and operations**
21. Makefile as the single entry point	25. Release packaging and checksums
22. Training-time validation and best-checkpoint keeping	26. Running stages on another machine (handoff bundles)
23. Data acquisition and pinned samples	27. Export, deployment and demo serving
24. Offline analysis from cached predictions	28. Hardware diagnostics and documenting decisions
Hardware limits	
29. Low-end GPUs and out-of-memory (OOM) playbook	

Part 1 — The Rules

1. Modularity

- Split code by responsibility, one concern per module: `configs/ YAML configs (see §5) data/ download, split building, dataset classes models/ model loading, adapters, architecture registry train/ training loop / trainer setup eval/ inference, scoring, metrics baselines/ third-party systems, same output format as our models scripts/ orchestration: queue runner, dashboard, boot hook, figures common.py shared constants and small helpers config.py path resolution (env vars → absolute paths)`
- Functions do one thing: `load_model()`, `load_examples()`, `predict_one()`, `compute_metrics()`, `write_results()`. Scripts are thin `main()` wrappers that parse arguments and call these functions.
- **New model or dataset = new registry entry, not a fork.** Keep a registry (dict of dataclasses) that holds the architecture-specific details (layer names, processor settings, `trust_remote_code`). Generic code reads the registry.
- New experiments go in **new files** that import existing helpers. Don't copy and edit a working script; extend it or import from it.
- One shared result format (e.g. `{"metrics": ..., "records": [...], "config": ...}`) for every system — models, baselines, ablations — so every downstream script (metrics, figures, bootstrap, dashboard) reads all of them unchanged.
- No logic at import time; everything runs under `if __name__ == "__main__":`.
- Import heavy libraries (`torch`, `transformers`) inside the functions that need them, so `--help`, `--status` and the dashboard start instantly and run without a GPU.

2. Cron jobs and process management

- Every multi-hour job gets an `@reboot` **crontab entry** that restarts it: `cron @reboot /home/<user>/<project>/scripts/resume_on_boot.sh`
- The boot script:
- uses absolute paths and sets `PATH` itself (cron has a minimal environment);
- exports the data-root environment variables;
- **waits for its dependencies** (data disk mounted, `nvidia-smi` succeeds) with a bounded timeout before starting;
- starts the dashboard if its port is free, then the runner;
- logs every start to `logs/boot.log` ;
- is safe to run by hand at any time (idempotent, see §3).
- Launch detached so a closed terminal or SSH drop doesn't kill the job: `setsid nohup python3 -u script.py >> logs/x.log 2>&1 < /dev/null &`
- Use `python3 -u` (or `PYTHONUNBUFFERED=1`) so logs appear in real time.
- **Kill processes by PID**, never with `pkill -f <pattern>` : the pattern also matches the shell running the command and kills it. Store the PID in the lock file, or use `pgrep -f` , look at the list, then `kill <pid>` .
- Periodic jobs (results sync, disk check, summary refresh) go in crontab with output redirected to a log; a cron line never writes to stdout.
- Document every cron entry in the README, including how to remove it.

3. Resumability

- **Training** checkpoints every N optimizer steps (≈30–60 min of work), keeps the last K checkpoints plus the best one, and auto-resumes from the latest on restart. The checkpoint holds optimizer, scheduler, RNG states and step count, not just weights.
- **Evaluation / inference** is resumable at item granularity:
- append one JSON line per finished item to `<output>.partial.jsonl` immediately after it is produced;
- on restart, load the partial file, skip completed IDs, continue;
- tolerate a truncated last line (process killed mid-write);
- when finished, write the final file **atomically** (write `x.tmp` , then `rename` to `x.json`) and delete the partial file.
- **Whole-run skip**: if the final output exists and is complete, the script exits immediately. Re-running a finished command costs nothing.
- **Downloads** skip files that already exist and are the right size/checksum; write to `file.part` and rename on completion.
- **Derived data** (splits, manifests, caches) is built deterministically from a seed, so rebuilding gives byte-identical output.
- A failure on one item (corrupt image, OOM) is **recorded as a failed record and the loop continues**, never a crash that loses hours of work. Catch OOM specifically, `torch.cuda.empty_cache()` , record, continue.
- **Queue runner** for many experiments:
- registry of experiments, ordered by priority;

- one subprocess per experiment (GPU memory fully released between runs);
- retry a failed experiment N times (each retry resumes), then move on and report it at the end;
- **lock file** (`fcntl.flock`) so two runners never share a GPU or append to the same partial file;
- `--status` , `--list` , `--only <names>` , `--group <group>` flags.
- **Test resumability** before the real run: start, kill it mid-way, restart, check that nothing is duplicated or lost.
- Handle `SIGTERM` / `SIGINT` gracefully: finish or discard the current item, flush files, exit.

4. Dashboard and monitoring

- Every long queue has a **status view**:
- CLI: `runner.py --status` prints done / partial (n/total) / pending per experiment with headline metrics;
- web: a small read-only dashboard (`http.server` from the standard library is enough) that auto-refreshes and has a JSON endpoint (`/api/status`).
- The dashboard shows: runner alive or not, GPU utilisation / memory / temperature, per-experiment progress bar, items/second, ETA, running metrics on items finished so far, failed-item count, "stalled" flag when no progress for N minutes, and the tail of the runner log.
- The dashboard **reads files only** (partial files, final outputs, logs). It never imports the training stack and never writes anything.
- Cache parsed files by (mtime, size) so refreshes stay cheap.
- Bind to `0.0.0.0` if you watch from another device; pick a port not used by other services and start it from the boot hook (§2).
- Log progress lines with rate and ETA (`...500/9000 1.9s/item eta 270 min`) every N items.
- Optionally push a notification (email, chat webhook, phone) when an experiment finishes or fails.

5. All configurations in YAML

- Every run is defined by a **YAML config file**, versioned in git under `configs/` . No hyperparameter lives only in a shell history or a command line. `yaml # configs/train/smolvlm2-2.2b_seed0.yaml name: smolvlm2-2.2b_seed0 model: id: HuggingFaceTB/SmolVLM2-2.2B-Instruct quantization: nf4 freeze_vision: true lora: {r: 16, alpha: 32, dropout: 0.05} data: train: ${DATA_ROOT}/instructions/train.jsonl val: ${DATA_ROOT}/instructions/val.jsonl train: epochs: 5 lr: 1.0e-4 batch_size: 1 grad_accum: 8 seed: 0 save_steps: 50 best_metric: m_acc output_dir: ${CHECKPOINT_ROOT}/smolvlm2-2.2b_seed0`
- Use **inheritance** to avoid copy-paste: a `base.yaml` with defaults, and small per-experiment files that override only what differs (`defaults:` / `_base_:` key, or OmegaConf/Hydra merge).
- Command-line overrides are allowed for quick tests (`--set train.lr=5e-5`), but the **resolved config is always saved** into the run's output directory (`config.resolved.yaml`) together with:
- git commit hash and `git diff` if the tree is dirty,
- library versions (`torch` , `transformers` , `peft` , CUDA),
- hashes or sizes of the data files used,
- hostname, GPU name, start time.
- The **experiment queue is also YAML** (`configs/queue.yaml`): list of experiments with name, group, priority, config path. The runner reads it.

- **Validate configs on load** (required keys, types, paths exist) and fail before loading any model.
- Names are derived from configs, and directory names never contradict the config (a directory called `-5ep` must not hold a 3-epoch run).

6. Data on the large disk

- Code lives in the repo (SSD); **data, checkpoints, caches and large results live on the big HDD**, e.g. `/mnt/bigdata/<project>/` .
- All paths resolve through **one config module** that reads environment variables with sensible defaults: `python DATA_ROOT = Path(os.environ.get("PROJECT_DATA_ROOT", "/mnt/bigdata/project/data")) CHECKPOINT_ROOT = Path(os.environ.get("PROJECT_CKPT_ROOT", "/mnt/bigdata/project/checkpoints"))` No absolute path is hard-coded anywhere else. `python3 config.py` prints the resolved paths.
- Machine-specific locations live in a `.env` **file** at the repo root, so moving to another machine or disk means editing one file: `dotenv # .env — machine-specific, gitignored. Copy from .env.example. PROJECT_DATA_ROOT=/mnt/bigdata/myproject/data PROJECT_CKPT_ROOT=/mnt/bigdata/myproject/checkpoints PROJECT_RESULTS_ROOT=/mnt/bigdata/myproject/results TB_ROOT=/mnt/bigdata/myproject/runs SHARED_MODELS_ROOT=/mnt/bigdata/models # cross-project model store (§20) HF_HOME=/mnt/bigdata/cache/huggingface # one HF cache for ALL projects TORCH_HOME=/mnt/bigdata/cache/torch PIP_CACHE_DIR=/mnt/bigdata/cache/pip TMPDIR=/mnt/bigdata/tmp DASHBOARD_PORT=8765 TB_PORT=6006 CUDA_VISIBLE_DEVICES=0`
- Commit a `.env.example` with the same keys and placeholder values; gitignore `.env` .
- **Precedence**: a variable already set in the environment beats `.env` , which beats the defaults in `config.py` . Load with `dotenv.load_dotenv(override=False)` at the top of `config.py` , before anything reads a path.
- Shell scripts and cron hooks don't read `.env` by themselves: load it explicitly with `set -a; source "$REPO/.env"; set +a` . This is the most common reason a job works by hand but fails from cron.
- Set cache variables (`HF_HOME` , `TORCH_HOME`) **before** importing the libraries, otherwise they are ignored.
- Secrets (`HF_TOKEN` , API keys, webhook URLs) go in a separate `.env.secrets` (gitignored, `chmod 600`) or the system keyring, never in `.env.example` , configs, logs or result metadata.
- `python3 config.py` prints every resolved path and whether it exists.
- Manifests store paths **relative to DATA_ROOT** (or are remapped on load), so moving the data doesn't break them.
- Point library caches at the big disk too: `HF_HOME` , `TORCH_HOME` , `PIP_CACHE_DIR` , `TMPDIR` for large temporary files.
- **Check that the data is reachable at startup**: if DATA_ROOT or the first image is missing, stop with a clear error. Never run a whole evaluation in which every item fails because a disk isn't mounted.
- Check free space before writing checkpoints or downloads; warn below a threshold.
- Never commit data, checkpoints or large results; `.gitignore` them. Commit small summaries (metrics without per-item records) instead.
- Keep the system disk for the OS and code; long jobs must not fill it.

7. Reproducibility

- Set seeds for `random`, `numpy`, `torch` (and CUDA) from the config; record the seed in every output.
- Pin dependencies (`requirements.txt` with versions, or a lock file).
- Deterministic split building from a seed; commit the split manifests (they are small) or their hashes.
- Every result file records: config, checkpoint path, data file, git commit, command line.
- Keep a **results ledger** (`results/LEDGER.md`): one line per result file with what produced it, so table numbers can be traced to files.

8. Logging

- One log file per experiment (`logs/<experiment>.log`) plus one for the runner and one for the boot hook.
- Every log entry has a timestamp; a run starts by logging its resolved config, GPU and data paths.
- Silence noisy, repeated library warnings once (e.g. with `transformers.logging.set_verbosity_error()`) so real errors stay visible.
- TensorBoard / W&B / CSV for training curves; the logger is configured in YAML.

9. GPU and memory hygiene

(Full low-memory and OOM playbook: §29.)

- Print free VRAM before loading a model and warn if other processes hold it (Ollama, ComfyUI, notebooks).
- One GPU job at a time unless the jobs are explicitly sized to share; the lock file enforces it.
- Release memory between models by using separate processes, not `del model`.
- Use `torch.no_grad()` / `inference_mode()` for evaluation; compute only the logits you need (`logits_to_keep=1`).
- Gradient checkpointing, 4-bit/8-bit loading and gradient accumulation are config switches, not code edits.

10. Testing before long runs

- **Smoke test** every configuration on ~20 items before a full run, for each model family (different architectures break in different places).
- Check that a known-good checkpoint reproduces its known result on the smoke subset.
- Any speed optimisation (batching, KV-cache reuse, packing) is **verified against the simple implementation** on real inputs with a stated tolerance, and the check runs automatically at the start of each job.
- Use `--limit N` (seeded random subset, not the first N lines, because files are often sorted by class) and `--dry-run` flags.
- Write smoke outputs to a temporary directory, never next to real results.

11. Results and file layout

```
configs/                                YAML: base, train/, eval/, queue.yaml,
figures.yaml
results/<experiment>/<name>.json        final outputs (atomic)
```

```
results/<experiment>/<name>.json.partial.jsonl   in-progress
results/<experiment>/<name>.predictions.parquet  per-item records (big disk)
results/summary/                          small, committed metric summaries
results/tables/                           generated LaTeX / CSV / Markdown tables
results/figures/<figure>.{pdf,png}        generated paper figures
results/figures/data/<figure>.csv         the exact numbers each figure plots
results/animations/                       GIF / MP4
results/grids/                            dataset and qualitative image grids
runs/<run>/                               TensorBoard event files
logs/<experiment>/<name>.log
checkpoints/<run>/config.resolved.yaml    + weights, optimizer state
```

- Never overwrite a finished result or checkpoint; new runs get new names.
- File names encode dataset, model, condition and seed: `cct_ft_2.2b_seed1.json`.
- Large per-item records stay on the big disk; summaries go to git.

12. Code hygiene

- Every script has a docstring with purpose and example usage, and `--help`.
- Fail early with a clear message (missing file, wrong config key, no GPU).
- No silent fallbacks that change results (e.g. falling back to CPU, to a different model, or to a default path); if a fallback exists, log it loudly.
- Commit working code before launching a multi-day run, so the run's commit hash points at the exact code.

13. TensorBoard

- Every training run writes TensorBoard events to `runs/<run_name>/` (on the big disk, symlinked or configured via `TB_ROOT`). The run name equals the config name, so TensorBoard runs, checkpoints and results line up.
- **Resume into the same directory** and pass the global step from the checkpoint, so a resumed run continues its curves instead of starting a new one or overwriting the old one.
- What to log:

Tag	What	When
`train/loss`, `train/lr`, `train/grad_norm`	scalars	every logging step
`train/samples_per_sec`, `sys/gpu_mem_gb`	throughput, peak memory	every logging step
`val/<metric>`	every validation metric incl. per-class	every eval step
`val/confusion_matrix`	rendered confusion-matrix image	every eval step
`val/pr_curve/<class>`	`add_pr_curve` per class	every eval step

Tag	What	When
`val/predictions`	fixed set of ~16 validation images with GT and prediction drawn on them	every eval step
`data/train_batch`	one augmented training batch as an image grid	once at start
`weights/<layer>`	histograms of trainable weights / adapters	sparsely
`embeddings`	`add_embedding` with thumbnails for the projector	end of training
`hparams`	`add_hparams(config, final_metrics)`	end of training
`text/config`	the resolved YAML config	once at start

- Use a **fixed, seeded set of validation images** for `val/predictions`, so you can watch the same images change over training.
- Log the config hash and git commit as text, so a curve can always be traced to its code.
- Start TensorBoard from the boot hook next to the dashboard: `tensorboard --logdir runs --host 0.0.0.0 --port 6006`. Link it from the dashboard page.
- With the Hugging Face `Trainer`, set `report_to=["tensorboard"]` and `logging_dir`, and add a callback for images, confusion matrices and PR curves, which the Trainer doesn't log by itself.
- TensorBoard is for watching training. Paper figures are made from saved result files (§14, §18), never from screenshots of TensorBoard.

14. Result saving

- Save **per-item predictions**, not just metrics. Every downstream analysis (new metric, bootstrap, per-class breakdown, failure gallery, operating curve) must be possible without re-running inference.
- One record per item contains: item ID, grouping keys (location / camera / patient / video), ground truth, prediction, **scores for every class** (logits, probabilities or log-likelihoods), confidence, raw model output (for generative models), failure/error field, latency. For detection/segmentation: boxes/masks with scores (COCO-format JSON or RLE masks), not only the matched results.
- Each result file carries a **metadata block**: resolved config, checkpoint path and hash, data file and hash, git commit, library versions, hostname, GPU, start and end time, command line.
- Formats:
- final result: JSON (`metrics` + `records` + `meta`), written atomically;
- large per-item tables: Parquet (fast, typed, compressed) or JSONL;
- in-progress: `.partial.jsonl` (§3);
- small committed summaries: metrics without records, in `results/summary/` ;
- tables for humans: generated CSV / Markdown / LaTeX in `results/tables/` .
- **Metrics are recomputable from records** by one function (`compute_metrics(records)`), shared by every system. Never compute metrics differently in different scripts.

- Save **uncertainty with every headline number**: bootstrap intervals (resampling grouped units such as locations, not single items), seed mean ± std.
- Save training histories (`trainer_state.json` / CSV of scalars) next to the checkpoint, so curves can be re-plotted without TensorBoard.
- **Never overwrite** a finished result. A rerun with a change gets a new name.
- Keep a ledger (`results/LEDGER.md`) mapping each result file to its config and command, and each paper number to its result file.

15. Visualizations

- **All figures come from scripts** (`scripts/make_figures.py` , or one module per figure under `viz/`) that read saved results. No figure is made by hand in a notebook and pasted.
- One function per figure: `fig_confusion(results, out_path)` . A `make figures` target regenerates every figure from result files in under a few minutes; if inputs are missing it skips that figure with a message instead of crashing.
- Every figure also writes the **numbers it plots** to `results/figures/data/<figure>.csv` , so values can be checked and quoted.
- Standard figure set for a vision project:

Figure	Shows
Training curves	loss, LR, val metric vs step; seeds as thin lines + mean
Confusion matrix	row-normalised, class counts in labels, sorted by frequency or by confusion clusters
Per-class bar chart	per-class accuracy/AP for each system, with class supports, sorted
PR / ROC curves	per class and macro; operating points marked
Operating curve	trade-off of two paired metrics (e.g. rejection vs recall) for every system, thresholds swept, chosen points marked
Reliability diagram	calibration: confidence vs accuracy, ECE in the legend
Scaling / ablation plot	metric vs model size, data size or a hyperparameter, with error bars
Embedding map	t-SNE / UMAP of features, coloured by class or by domain (shows domain shift)
Attribution maps	Grad-CAM / attention rollout / attention over frames overlaid on images
Qualitative grid	correct and failure cases with GT and prediction (§16)
Failure gallery	top-confidence errors, grouped by error type
Domain-shift panel	same classes from source and target datasets side by side, with metrics
Dataset statistics	class histogram (log scale), items per location/camera, image sizes, day/night split

- **Colour is consistent across all figures**: each system/model has one fixed colour and marker defined in one place (`viz/style.py`); classes get a fixed palette. Use colour-blind-safe palettes (Okabe-Ito, Tol, viridis for sequential, a diverging map centred on zero for differences).
- Show uncertainty: error bars or shaded bands from bootstrap or seeds, with the interval type stated in the caption.
- Interactive versions (Plotly/HTML) are fine for exploration and the dashboard; paper versions are static vector files.

16. Dataset grids

- Build image grids **with code**, deterministically (seeded sampling), and save both the image and a JSON listing the item IDs shown, so a grid can be regenerated or cited.
- Grids to make for every vision dataset:
- **Class × samples**: one row per class, N random samples per row, class name and count at the row start. First thing to look at for a new dataset.
- **Per split / per domain**: the same class from train, test and the transfer dataset side by side (shows what "held-out location" or "domain shift" looks like).
- **Per location / camera**: one tile per camera, showing backgrounds (reveals background leakage).
- **Hard cases**: empty/background frames, night/IR, occlusion, blur, tiny objects, multiple objects.
- **Clip/sequence strips**: all frames of one sample in a row (for video, multi-frame or burst inputs).
- **Augmentation preview**: one image under each augmentation.
- **Annotation check**: boxes/masks drawn on images, random sample, to catch label errors before training.
- Qualitative result grids:
- border or label colour encodes correctness (green correct, red wrong, orange low confidence), text shows `GT → prediction (score)` ;
- columns = systems, rows = the same items, so systems are compared on identical inputs;
- pick items by rule (random seeded, or top-k confident errors), never by hand-picking the nicest ones, and state the rule in the caption.
- Implementation:
- resize to a fixed thumbnail size with `PIL.ImageOps.fit` / letterbox, keep aspect ratio, consistent padding (2–4 px) and background;
- draw text with a readable font and a contrasting backing box;
- cache thumbnails on the big disk so regenerating grids is fast;
- output PNG for the web, PDF/high-DPI PNG (≥300 dpi at print size) for the paper; check file size (paper PDFs have limits — use JPEG-compressed thumbnails inside the PDF if needed).
- Respect the dataset's licence and privacy rules: blur people/faces and licence plates before publishing grids.

17. Animations

- Make animations with code (`imageio` / `ffmpeg` / `matplotlib.animation`), write them to `results/animations/` , and save the frame list/seed alongside.
- Useful animations for vision projects:

- **Sample playback**: the frames of a clip/video with GT, prediction and confidence overlaid per frame.
- **Prediction evolution**: the same fixed validation images across training checkpoints, showing predictions/attention changing.
- **Attention / saliency over time**: Grad-CAM or attention maps per frame, overlaid with transparency.
- **Detection/tracking**: boxes and track IDs drawn frame by frame.
- **Threshold sweep**: an operating curve with a moving point and the corresponding accepted/rejected images.
- **Embedding drift**: t-SNE/UMAP of features across training checkpoints (fixed seed and aligned axes).
- **Dataset tour**: short montage of the class grid for talks and README.
- Formats: MP4 (H.264, `yuv420p`, even width/height) for slides and web; GIF only when small (palette-optimised, ≤ ~10 MB); WebM for the project page. Keep a static frame (PNG) for the paper, since PDFs cannot show video.
- Fixed frame rate, fixed resolution, a title card or legend, and no frame over 1–2 s unless it is a pause on a key result.

18. CVPR-style paper figures and tables

- **Size figures to the layout.** CVPR/ICCV/ECCV two-column: single column ≈ 3.25 in (8.25 cm), full width ≈ 6.875 in (17.5 cm). Create figures at the final size (`figsize=(3.25, 2.2)`) so fonts don't get scaled down.
- **Fonts match the paper**: 7–9 pt in figures (the body text is 10 pt), Times or the template font, never smaller than the caption font. Set them once: `python # viz/style.py import matplotlib as mpl mpl.rcParams.update({ "font.family": "serif", "font.serif": ["Times New Roman", "Times", "DejaVu Serif"], "font.size": 8, "axes.labelsize": 8, "axes.titlesize": 8, "legend.fontsize": 7, "xtick.labelsize": 7, "ytick.labelsize": 7, "axes.linewidth": 0.6, "lines.linewidth": 1.2, "lines.markersize": 4, "axes.spines.top": False, "axes.spines.right": False, "pdf.fonttype": 42, "ps.fonttype": 42, # embed TrueType, required by many venues "savefig.bbox": "tight", "savefig.pad_inches": 0.02, }) SYSTEM_STYLE = { # one colour/marker per system, used by every figure "ours": {"color": "#0072B2", "marker": "o"}, "baseline": {"color": "#D55E00", "marker": "s"}, }`
- **Vector output** (PDF) for plots; raster (PNG/JPEG at ≥300 dpi) only for photos inside grids. Save PDF and PNG for every figure.
- No figure titles — the caption does that. Axis labels carry units; legends do not cover data; label lines directly when there are few.
- **Teaser figure (Fig. 1)**: one full-width figure that shows the task, the method and the key result at a glance (input examples → system → output, plus the headline number or trade-off).
- **Method diagram**: drawn in a vector tool (draw.io, Inkscape, TikZ, PowerPoint exported to PDF), same colours as the plots, saved in `paper/figures/src/` with the source file.
- **Multi-panel figures**: panels labelled (a), (b), (c) in a consistent position; shared axes where comparable; build with `plt.subplots` / `subfigures` or LaTeX `subcaption`, not by pasting images.

- **Tables are generated** from result files into LaTeX (`booktabs` : `\toprule \midrule \bottomrule` , no vertical lines), with best result in bold and second underlined, ± intervals or a CI column, and units in the header. The paper `\input{}` s the generated `.tex` , so a rerun updates the paper.
- Every number shown in a figure or table is traceable to a result file via `results/figures/data/*.csv` and the ledger.
- Check before submission: readable when printed in greyscale, readable at 100% zoom, consistent naming of systems across figures, tables and text, file sizes within the venue's limit.

19. Garbage collection and teardown

Long-running projects leave things behind: cron entries that restart a finished queue on every boot, dashboards still bound to ports, orphaned processes holding GPU memory, stale lock and partial files, gigabytes of intermediate checkpoints and caches. The principle: **anything the project installs or starts, the project can list and remove — and only its own things.**

19.1 SAFETY RULES FOR EVERY CLEANUP ACTION

- **Dry run by default.** Every cleanup tool prints what it *would* do; deleting needs an explicit `--apply` .
- **Look before deleting.** List the targets with sizes and ages, and confirm that nothing in the list is still in use or referenced.
- **Only inside allowed roots.** Cleanup code checks that every path it touches is under a configured root (`DATA_ROOT` , `CHECKPOINT_ROOT` , `RESULTS_ROOT` , a scratch dir) after resolving symlinks. Refuse anything else.
- **Never** `rm -rf "$VAR/..."` **with a possibly empty variable.** In shell use `set -euo pipefail` and `"${VAR:?not set}"` ; in Python, build paths with `pathlib` and assert `path.is_relative_to(root)` .
- **Move to trash, then expire.** Results and checkpoints move to `<root>/_trash/<YYYY-MM-DD>/` first; a later GC pass deletes trash older than N days. Only regenerable files (caches, `__pycache__` , temp files) are deleted directly.
- **Protected list.** Anything referenced by the results ledger, the paper or a `KEEP` file is never touched by automatic cleanup.
- **Log every deletion** (path, size, reason, timestamp) to `logs/gc.log` .
- **Back up before editing system state** (crontab, systemd units).

19.2 CRON ENTRIES: TAG, LIST, EXPIRE, PRUNE

Untagged cron lines can't be told apart later, and they outlive the project that added them. Rules:

- Every entry a project installs gets a **marker comment on the line above** with the project, a job ID, an expiry date and the script it runs: `cron # managed-by=wildlife-vlm id=resume-queue expires=2026-11-30 @reboot /home/roy/wildlife-vlm/scripts/resume_on_boot.sh`
- Install through a tool, never by hand-editing, so installing twice **replaces** the entry instead of duplicating it.
- Prune entries that are **expired**, whose **script no longer exists**, or whose job reports itself finished.
- Never modify **unmanaged** entries; list them so you can decide manually.

- **Self-retiring jobs**: a boot hook for a queue checks whether all work is done; if so it removes its own cron entry and exits. `bash if python3 scripts/run_queue.py --all-done; then python3 ops/cronctl.py remove --project wildlife-vlm --id resume-queue --apply exit 0 fi`

A universal cron manager — copy it into any project as `ops/cronctl.py`:

```
#!/usr/bin/env python3
"""Manage tagged crontab entries: install / list / remove / prune.
Only touches entries tagged '# managed-by=<project> id=<id> expires=<date|never>'.
Dry run unless --apply. Backs up the crontab before every write."""
import argparse, datetime as dt, os, re, shlex, subprocess, sys
from pathlib import Path

MARK = re.compile(r"^# managed-by=(\S+) id=(\S+) expires=(\S+)\s*$")
BACKUP_DIR = Path.home() / ".crontab-backups"

def read():
    r = subprocess.run(["crontab", "-l"], capture_output=True, text=True)
    return r.stdout.splitlines() if r.returncode == 0 else []

def write(lines):
    BACKUP_DIR.mkdir(exist_ok=True)
    stamp = dt.datetime.now().strftime("%Y%m%d-%H%M%S-%f")  # unique per write
    (BACKUP_DIR / f"crontab.{stamp}").write_text("\n".join(read()) + "\n")
    subprocess.run(["crontab", "-"], input="\n".join(lines) + "\n", text=True, check=True)

def parse(lines):
    """-> list of blocks: {'managed': bool, 'project', 'id', 'expires', 'lines'}"""
    blocks, i = [], 0
    while i < len(lines):
        m = MARK.match(lines[i])
        if m and i + 1 < len(lines):
            blocks.append({"managed": True, "project": m[1], "id": m[2], "expires": m[3],
                           "lines": lines[i:i + 2]})
            i += 2
        else:
            blocks.append({"managed": False, "lines": [lines[i]]})
            i += 1
    return blocks

def entry_script(entry):
    """First absolute path in the command part of a cron line, if any."""
    parts = entry.split(None, 1 if entry.startswith("@") else 5)
    cmd = parts[-1] if parts else ""
    for tok in shlex.split(cmd, posix=True) if cmd else []:
        if tok.startswith("/"):
            return tok
    return None

def stale_reason(b, today):
    if b["expires"] != "never" and dt.date.fromisoformat(b["expires"]) < today:
        return f"expired {b['expires']}"
    script = entry_script(b["lines"][1])
    if script and not os.path.exists(script):
        return f"missing {script}"
    return None
```

```
def main():
    p = argparse.ArgumentParser(description=__doc__)
    sub = p.add_subparsers(dest="cmd", required=True)
    for name in ("install", "remove", "prune", "list"):
        s = sub.add_parser(name)
        s.add_argument("--project")
        s.add_argument("--apply", action="store_true")
        if name in ("install", "remove"):
            s.add_argument("--id", required=(name == "install"))
        if name == "install":
            s.add_argument("--schedule", required=True, help="'@reboot' or '*/30 * * * *'")
            s.add_argument("--command", required=True)
            s.add_argument("--expires", default="never", help="YYYY-MM-DD or never")
    a = p.parse_args()
    blocks, today = parse(read()), dt.date.today()

    def mine(b):
        return b["managed"] and (a.project is None or b["project"] == a.project)

    if a.cmd == "list":
        for b in blocks:
            if b["managed"] and mine(b):
                why = stale_reason(b, today)
                print(f"[{b['project']}/{b['id']}] expires={b['expires']}"
                      f"{'  STALE: ' + why if why else ''}\n    {b['lines'][1]}")
            elif not b["managed"] and b["lines"][0].strip() and not b["lines"]
[0].startswith("#"):
                print(f"[unmanaged] {b['lines'][0]}")
        return

    if a.cmd in ("install", "remove") and not a.project:
        sys.exit("--project is required")   # never remove every project's entries by
accident
    if a.cmd == "install":
        keep = [b for b in blocks if not (mine(b) and b["id"] == a.id)]
        new = [f"# managed-by={a.project} id={a.id} expires={a.expires}",
               f"{a.schedule} {a.command}"]
        print("install:\n    " + "\n    ".join(new))
    else:  # remove / prune
        def drop(b):
            if not mine(b):
                return None
            if a.cmd == "remove":
                return "removed" if (a.id is None or b["id"] == a.id) else None
            return stale_reason(b, today)
        keep, new = [], []
        for b in blocks:
            why = drop(b)
            if why:
                print(f"{'remove' if a.apply else 'would remove'} [{b['project']}/{b['id']}]
({why}): {b['lines'][1]}")
            else:
                keep.append(b)

    lines = [l for b in keep for l in b["lines"]] + new
    if a.apply:
        write(lines)
```

```
        print("crontab updated (backup in ~/.crontab-backups)")
    else:
        print("dry run — add --apply to write")

if __name__ == "__main__":
    main()
```

Usage:

```
python3 ops/cronctl.py install --project myproj --id resume-queue \
    --schedule @reboot --command "$PWD/scripts/resume_on_boot.sh" --expires 2026-12-31 --
apply
python3 ops/cronctl.py list                      # every project + unmanaged lines, stale
ones flagged
python3 ops/cronctl.py prune --apply             # drop expired / missing-script entries
(all projects)
python3 ops/cronctl.py remove --project myproj --apply   # project teardown
```

Run `cronctl.py prune` from a weekly GC job (19.9) so stale entries from any project disappear on their own.

19.3 PROCESSES, PORTS AND GPU MEMORY

- Every long-lived process writes a **PID file** (or puts its PID in its lock file) under `logs/`, together with its start time and command.
- Before killing a PID, **check it is still the process you think it is**: PIDs are reused after reboot. Compare `/proc/<pid>/cmdline` (or `ps -o args= -p <pid>`) with the expected command. Then `SIGTERM`, wait (e.g. 30 s for the current item to finish), `SIGKILL` only if needed.
- Kill by PID, never by `pkill -f <pattern>` (it matches your own shell and unrelated processes).
- **Orphans**: if the runner is dead but a child (`score.py`, `train.py`) is alive, or vice versa, the GC reports it. A queue runner kills its child on `SIGTERM` (forward the signal) so this doesn't happen in normal operation.
- **GPU leftovers**: `nvidia-smi --query-compute-apps=pid,process_name,used_memory --format=csv` lists processes holding VRAM. Typical culprits: dead Jupyter kernels, inference servers keeping models resident (e.g. `ollama ps` / `ollama stop <model>`), image-generation UIs, crashed training runs. Report them before starting a job; stop only what you own.
- **Ports**: `ss -ltnp` shows what is listening. Every project records its ports (dashboard, TensorBoard) in its config; teardown stops those servers. Don't leave dashboards bound to `0.0.0.0` after the project ends.
- `tmux` / `screen` sessions are named after the project (`tmux new -s myproj-queue`) so they can be found and closed.

19.4 LOCK FILES AND PARTIAL FILES

- Prefer `flock`**-based locks**: the kernel releases them when the process dies, so they can never go stale. An existing lock *file* means nothing; only a held lock does.
- If a project uses PID-in-file locks, a lock is stale when its PID is dead or belongs to another command; GC removes such locks.
- **Partial result files** (`*.partial.jsonl`):

- belonging to a queued experiment → never touched (they are the resume state);
- belonging to an experiment no longer in any registry/config → reported as orphaned, moved to trash after confirmation;
- older than N days with no progress → reported as abandoned.
- **Temporary files** from interrupted atomic writes (`*.tmp` , `*.part`) older than a day are deleted.
- **Smoke-test outputs** live in a scratch directory and are deleted when the smoke test passes; they never sit next to real results.

19.5 CHECKPOINTS

- Retention policy, set in YAML: `yaml retention: keep_last: 2 # latest checkpoints for resume keep_best: 1 # by the configured selection metric keep_referenced: true # anything named in the ledger / paper / KEEP file strip_optimizer_after_finish: true trash_days: 14`
- **During training**: `save_total_limit` keeps the last K checkpoints; the best checkpoint is preserved separately.
- **After a run finishes**: delete optimizer/scheduler states from all but the final checkpoint (they are often 2–3× the size of the weights), and move intermediate checkpoints not covered by the policy to trash.
- **Abandoned runs** (no progress for N days, not in the queue) are listed for a decision, never deleted automatically.
- Merged/exported models (GGUF, ONNX, merged fp16) are regenerable; keep the adapter + config and delete exports you don't ship.

19.6 CACHES AND DOWNLOADS

Cache	Inspect	Clean
Hugging Face models/datasets	`hf cache scan` (or `huggingface-cli scan-cache`)	`hf cache delete` — remove models no config references
pip	`pip cache info`	`pip cache purge`
PyTorch hub / timm	`du -sh ~/.cache/torch`	delete unused model dirs
Inference servers (e.g. Ollama)	`ollama list`	`ollama rm <model>` for unused models
Docker	`docker system df`	`docker system prune` (review first; `-a` removes all unused images)
Thumbnails / feature caches	`du -sh <cache root>`	delete — regenerable
`__pycache__` , `.pytest_cache`	—	`find . -name __pycache__ -prune -exec rm -rf {} +`
Downloaded archives already extracted	list `*.zip` , `*.tar.gz` next to extracted dirs	delete once extraction is verified (keep checksums)

- Point caches at the big disk (§6) so they never fill the system disk.
- Before deleting a model from a cache, grep the configs for its ID.

19.7 LOGS AND TENSORBOARD RUNS

- Rotate logs by size (e.g. 50 MB) with `logrotate` or `logging.handlers.RotatingFileHandler`; compress old ones; keep N rotations.
- Silence warnings that repeat per item; one warning per run is enough. A log that is 99% repeated warnings hides the real error.
- TensorBoard: delete runs from smoke tests and aborted configurations; keep runs referenced by results. Remove runs by directory, never by editing event files.

19.8 DISK-SPACE WATCHDOG

- A cron job every 30–60 min checks free space on each root and writes a warning to the dashboard / sends a notification below a threshold (e.g. 10% or 50 GB).
- Long jobs check free space before writing checkpoints and pause with a clear message instead of failing halfway through a write.
- `du -sh <root>/* | sort -h` and `ncdu` find what grew.

19.9 A UNIVERSAL GC TOOL

Give every project one entry point, `ops/gc.py`, with subcommands that share the safety rules (19.1):

Command	Does
`gc.py audit`	report everything below: cron (managed / stale / unmanaged), processes and orphans, GPU holders, listening ports, locks, partial/tmp files, checkpoint sizes vs policy, caches, log sizes, trash, free disk — no changes
`gc.py cron`	`cronctl.py prune`
`gc.py procs`	stale PID files, orphans; stop own processes with `--apply`
`gc.py files`	tmp/part files, orphaned partials, smoke outputs
`gc.py ckpt`	apply the retention policy
`gc.py caches`	list unreferenced cached models and regenerable caches
`gc.py trash`	empty trash older than `trash_days`
`gc.py teardown`	full end-of-project cleanup (19.10)

Install `gc.py audit` as a **weekly managed cron job** that only writes a report (`logs/gc-report-<date>.md` , shown on the dashboard). Deleting stays a manual `--apply` step, except for regenerable files and expired cron entries.

19.10 END-OF-PROJECT TEARDOWN CHECKLIST

1. Results: final result files, summaries, tables and figures exist and are in the ledger; summaries committed.
2. Archive what must be kept (configs, best checkpoints/adapters, per-item predictions, ledger) to long-term storage and verify checksums.
3. `cronctl.py remove --project <name> --apply` , then `cronctl.py list` to confirm.
4. Stop dashboards, TensorBoard and runners (by PID, verified); check `ss -ltnp` and `nvidia-smi` are clean of project processes.
5. Remove systemd user units / tmux sessions the project created, if any.
6. Apply the checkpoint retention policy; strip optimizer states.
7. Remove orphaned partial, tmp and smoke files; empty project trash.
8. Remove cached models no other project uses; remove unneeded downloaded archives.
9. Remove project environment variables from shell profiles if they were added.
10. Write a short `TEARDOWN.md` : what was archived where, what was deleted, what was kept on the big disk and why.

20. Hugging Face rules and the shared model store

20.1 LOADING MODELS AND PROCESSORS

- Every model is specified in YAML by **ID + pinned revision** (a commit hash, not `main`), so a model update upstream can't silently change results: `yaml model: id: HuggingFaceTB/SmolVLM2-2.2B-Instruct revision: <commit-sha> # from the model page "Files and versions" dtype: bfloat16 attn_implementation: sdpa trust_remote_code: false quantization: {load_in_4bit: true, quant_type: nf4, double_quant: true, compute_dtype: bfloat16} processor_kwargs: {} # e.g. image tiling caps`
- Load through one function (`models/load.py::load_model(cfg)`) that builds `BitsAndBytesConfig` , calls `AutoModelFor…` / `AutoProcessor` with the same `revision` , and logs the resolved commit (`model.config._commit_hash`) into the run metadata.
- `trust_remote_code=True` only for models in the registry, always with a pinned revision (remote code runs on your machine).
- Set `dtype` , `device_map` and `attn_implementation` explicitly; don't rely on library defaults that change between versions.
- Use the **model's own chat template** through `processor.apply_chat_template` ; never hand-build prompt strings for chat models. Log the rendered template once per run.
- Silence repeated library warnings once (`transformers.logging.set_verbosity_error()`), after checking them once.

20.2 CACHES, DOWNLOADS AND OFFLINE RUNS

- **One shared cache for all projects** on the big disk (`HF_HOME` in `.env`), so the same 5 GB model is not downloaded once per project. (`HF_HUB_CACHE` / `HF_DATASETS_CACHE` can split hub files and datasets.)

- A **setup step pre-downloads** every model and dataset in the configs (`huggingface_hub.snapshot_download(repo_id, revision=…)`), then long runs use `HF_HUB_OFFLINE=1` : a network hiccup at hour 30 can't fail a `from_pretrained` call, and nothing is fetched behind your back.
- Authenticate with `hf auth login` or `HF_TOKEN` from `.env.secrets` ; never put tokens in code or configs.
- Respect licences and gated-model terms; record the licence of every model and dataset in the run metadata and model cards.

20.3 DATASETS LIBRARY

- `load_dataset(name, revision=…)` with a pinned revision; cache on the big disk.
- Expensive preprocessing (`.map`) runs once, with `num_proc` and a fixed `fingerprint` / `cache_file_name` , and the result is saved with `save_to_disk` to `DATA_ROOT/processed/<name>/<version>/` ; training loads that.
- Use streaming (`streaming=True`) for datasets larger than the disk.
- Keep IDs and grouping keys (location, camera, video) as columns; they are needed for leak-free splits and grouped bootstrap.

20.4 TRAINER AND PEFT RULES

- `TrainingArguments` are built **from the YAML config**, never typed into code.
- `output_dir` on the big disk; `save_strategy="steps"` , `save_steps` , `save_total_limit` ; `eval_strategy` matching `save_strategy` when using `load_best_model_at_end` with `metric_for_best_model` .
- **Auto-resume**: `trainer.train(resume_from_checkpoint=get_last_checkpoint(output_dir))` (from `transformers.trainer_utils`), so the same command starts or resumes.
- `report_to=["tensorboard"]` , `logging_dir=TB_ROOT/<run>` (§13); `seed` and `data_seed` from config.
- Vision/VLM specifics: `remove_unused_columns=False` (keeps `pixel_values`); a custom collator that masks prompt and padding tokens to `-100` so loss is computed on the answer only; `gradient_checkpointing_kwargs={"use_reentrant": False}` .
- PEFT: target modules come from the architecture registry or are discovered and **logged**; print `model.print_trainable_parameters()` into the run log; `prepare_model_for_kbit_training` for k-bit models.
- Check that the adapter's `base_model_name_or_path` is the **hub ID** (not a local temp path), so the adapter can be loaded on another machine.

20.5 SAVING A MODEL SO IT'S REUSABLE

A saved model directory must be **self-contained**: someone in another project can load it with only the directory.

- `model.save_pretrained(dir, safe_serialization=True)` (safetensors, never pickle `.bin`) **and** `processor.save_pretrained(dir)` / tokenizer in the same directory, including the chat template and any processor kwargs.
- `config.json` carries `id2label` / `label2id` (classification) or the label vocabulary file next to the weights, so class names travel with the model.

- Also save: `config.resolved.yaml` (training config), `metrics.json` (validation/test metrics with intervals), `training_args.json`, `requirements.txt` / library versions, and a **model card** (`README.md`) with: base model and revision, task, label set, training data and splits, recipe, metrics, intended use, limitations, licence, source project and git commit.
- For PEFT, store the adapter (small) and optionally a merged fp16 copy (`merge_and_unload()`) for users without `peft`. Exports (GGUF, ONNX, TorchScript) go in an `exports/` subfolder with the command used.

20.6 THE SHARED MODEL STORE

Trained models that other projects may reuse are **published** from the project's checkpoint area to one shared location on the big disk (`SHARED_MODELS_ROOT`, e.g. `/mnt/bigdata/models`):

```
/mnt/bigdata/models/
  index.yaml                                  # registry of everything in the store
  <namespace>/<model-name>/
    v1/                                       # immutable once written
      adapter_model.safetensors | model.safetensors
      adapter_config.json | config.json
      processor + tokenizer files, chat_template.jinja
      README.md  (model card)
      config.resolved.yaml  metrics.json  training_args.json
      CHECKSUMS.sha256
    v2/
    latest -> v2                              # symlink, the only mutable thing
```

- **Versions are immutable.** A retrained model is a new version directory; `latest` is moved only by the publish script.
- `index.yaml` lists every entry: name, version, base model + revision, task, labels, headline metrics, path, date, source project, git commit, checkpoint it came from, licence.
- **Publish with a script**, never by hand: `scripts/publish_model.py --checkpoint <dir> --name wildlife/smolvlm2-2.2b-cct --version v1` It copies the selected checkpoint, writes the model card, metrics and checksums, test-loads the result in a fresh process, appends to `index.yaml`, and refuses to overwrite an existing version.
- **Load from any project** through a tiny helper (vendored into each project): `python def load_shared(name, version="latest"): root = Path(os.environ["SHARED_MODELS_ROOT"]) / name / version cfg = yaml.safe_load((root / "config.resolved.yaml").read_text()) if (root / "adapter_config.json").exists(): base = AutoModelForImageTextToText.from_pretrained(cfg["model"]["id"], revision=cfg["model"].get("revision")) model = PeftModel.from_pretrained(base, root) else: model = AutoModelForImageTextToText.from_pretrained(root) return model, AutoProcessor.from_pretrained(root)` Consumers pin a version (`v2`), not `latest`, in their own YAML.
- Permissions: the store is group-readable (`chmod -R g+rX`), written only by the publish script.
- Optional off-machine backup: also `push_to_hub(..., private=True)` to a private Hub repo, recording the repo ID and commit in `index.yaml`.
- The store is covered by GC protection (§19.1): nothing in it is deleted by automatic cleanup; retiring a version is an explicit, logged action that updates `index.yaml`.

21. Makefile as the single entry point

- One `Makefile` exposes every pipeline stage as a target: `setup`, `check`, `data`, `train`, `eval`, `baselines`, `analysis`, `figures`, `smoke`, `release`, `clean-derived`, and `all` chaining them in order.
- **Self-documenting**: each target has a `## description` comment and `make help` (the default goal) prints them: `make .DEFAULT_GOAL := help help: ## Show this help @grep -E '^[a-zA-Z_-]+:.*?## .*$$' $(MAKEFILE_LIST) \ | awk 'BEGIN{FS=":.*?## "}{printf "\033[36m%-16s\033[0m %s\n", $$1, $$2}'`
- Variables with `?=` defaults are overridable from the command line, so a new model or run is `make train eval MODEL=<id> RUN=<name>`, not an edited file.
- The Makefile **asks** `config.py` **for paths** (`DATA_ROOT ?= $(shell python3 -c "from config import DATA_ROOT; print(DATA_ROOT)")`) instead of duplicating them, so there is one source of truth.
- `make check` prints resolved paths and verifies imports and CUDA, the first thing to run on a new machine.
- Every target is resumable and safe to re-run (§3), so `make all` after an interruption continues the pipeline.
- `make clean-derived` removes **only regenerable** artifacts (splits, instruction files, figures), never raw data, checkpoints or results.
- `docs/reproducing.md` lists each target with expected wall-clock time and whether it needs a GPU.

22. Training-time validation and best-checkpoint keeping

- **Validate on the task metric, not only the loss.** A training callback runs real inference on a validation subset at every checkpoint and logs accuracy, macro accuracy, per-class/background recall and the invalid-output rate. Loss can look healthy while the model collapses to always predicting the majority class or "empty"; only task metrics show that.
- The validation subset is **seeded random, not the first N lines** (files are often sorted by class; a prefix can be 100% one class).
- Make it large enough to rank checkpoints; a few dozen items gives noisy selection. If time forces a small subset, re-evaluate the top few checkpoints on the full validation split before choosing.
- Validation metrics go to three places: console, an append-only `val_metrics.log` in the run directory, and TensorBoard. **NaN-safe**: skip NaN values when logging (a subset with no positives gives NaN), and never let NaN count as an improvement.
- **Preserve the best checkpoint outside the rotation.** `save_total_limit` deletes old checkpoints, including an early peak. Copy every new best into `<run>/best_checkpoint/` with a `meta.json` (step, metric, value) and the run config, so the best model survives and is self-contained.
- Every intermediate checkpoint is a **directly usable model** (e.g. a valid adapter directory), so progress can be evaluated before training ends.
- Decide in advance — and write into the config — which checkpoint the paper reports (final or best-on-validation), and use that same one for every test set.
- Memory settings that prevent OOM restarts:
- set `PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True` **before importing torch** (reduces fragmentation OOMs on long runs);

- when inputs have variable size (image tiling, multi-frame clips), use batch size 1 + gradient accumulation; measure peak memory on the largest input before choosing the batch size;
- checkpoint often enough that an OOM or crash loses minutes, not hours.

23. Data acquisition and pinned samples

- **Parallel, resumable downloads**: a thread pool (8–16 workers), skip files that exist with non-zero size, delete the partial file on any failure so it isn't mistaken for a valid one, catch `OSError` (covers resets, timeouts, SSL errors) per file instead of crashing the run, print progress every N files, and report the failure count at the end so a second pass can fetch the rest.
- Keep source URLs as named constants at the top of the download script, with a comment saying where to find current links if they move.
- A **development-scale cap** (`--max-items-per-split N`) lets the whole pipeline run on a small sample first. Sample-scale and full-scale data live in **separate directories** (`splits_sample3k/` , `splits_full/`) and every result file is tagged with its scale in the ledger, so the two are never mixed in one table.
- When a dataset is too large to download fully, draw a **seeded, stratified sample** and **pin it in git**: the selected item list, the exact URL list and a manifest (seed, method, per-class counts, source file version). The sample then reproduces without the huge annotation file; re-sampling is a separate, explicit command.
- **Verify assumptions about a dataset by inspecting the real files**, and write the result and date into the code comment (e.g. "0 of 14k train annotations are empty — checked 2026-08-08"). Assumptions taken from papers or docs are often wrong.
- Write data-format code against the **common schema** (e.g. COCO / COCO Camera Traps), so other datasets with the same schema work unchanged.
- Map each external dataset's labels onto your vocabulary with an explicit, committed mapping table; classes with no counterpart are excluded from scoring and listed.
- Paths embedded in manifests are remapped on load (§6), so manifests built on one machine work on another.

24. Offline analysis from cached predictions

- Separate **inference** (GPU, slow) from **analysis** (CPU, fast). Every analysis script — metrics, operating curves, bootstrap, re-scoring, comparison tables, figures — reads saved result files and runs without a GPU or a model.
- Keep raw model outputs, so new parsing rules can be applied later: e.g. a **strict** parser (the official metric) and a **lenient** re-scorer that extracts labels from free text, reported side by side to separate "can't recognise" from "won't follow the format". Note the re-scorer's known over-crediting cases in its output.
- Third-party baselines are scored by **the same metric code** on the same items, from their released prediction files when available (no need to re-run them).
- A summarizer strips per-item records into small **committed summaries** (`results/summary/`); figures and tables read summaries first and fall back to full files, so a fresh clone regenerates aggregate figures with no GPU and no downloads. Figures needing per-item data or images are skipped with a message when those are absent.

- Comparison tables against published numbers read a hand-transcribed JSON of the paper's values, stored in the repo with the source (paper, table number).

25. Release packaging and checksums

- Large artifacts (checkpoints, per-item results, prepared splits, rendered figures) are packaged by a script (`scripts/make_release.py`) into tarballs for a GitHub Release / Zenodo / institutional storage, never committed to git.
- **Inference-ready packages**: strip optimizer and scheduler states, which are only needed to resume training and are often most of the checkpoint size.
- Write `SHA256SUMS.txt` for every asset and `RELEASE_NOTES.md` describing each asset, its size, and how to use it (`sha256sum -c SHA256SUMS.txt`).
- Support building one asset at a time (`--only results`).
- Releases say what is regenerable and which command regenerates it.

26. Running stages on another machine (handoff bundles)

When a second machine (lab server, friend's GPU, cloud VM) runs part of the queue:

- Build a **self-contained bundle directory**: code, the exact checkpoint, the eval files with paths relative to the bundle, the label list, and one `run_stage.sh` .
- `run_stage.sh` uses `set -euo pipefail` , checks its inputs exist with a clear message, is resumable (same partial-file mechanism), and prints at the end which files to copy back.
- A `SWAP_IN.md` explains what to copy where, and what the stage produces.
- For a sequence of stages, ship **delta bundles** (only the new checkpoint and script) when the code and data are already on the other machine.
- Result files carry their metadata (§14), so results coming back from another machine are traceable; copy them into `results/` and record the machine in the ledger.
- Pack images as one tar (not thousands of small files) for transfer.

27. Export, deployment and demo serving

- **Export pipeline as a script with stages**: merge adapters into the base model (on CPU by default, since the GPU is usually busy), convert (GGUF / ONNX), quantize, and benchmark. Each stage is a separate command that can be re-run alone; a preflight step reports which tools (compiler, cmake, converter) are available before doing any work; every setting is overridable by environment variable, so the same script exports any model.
- **Benchmark on the target device**, with a small, per-class, seeded benchmark bundle: a `manifest.json` the device reads (inputs only, **no labels**) and a server-side `groundtruth.json` for scoring, so labels never reach the client being tested.
- **Demo UI / API server** for qualitative checks:
- endpoints to list models, fetch a random real test item with its label, and run a prediction;
- auto-discovers the best checkpoint (`best_checkpoint/` before the run root);
- loads a GPU model only when asked, and says in its docs that it holds VRAM, so it isn't started next to training;
- can talk to a local inference server (e.g. Ollama) for baseline models without using the training GPU's memory.

- Inference servers are also a way to run **baseline evaluations while the GPU trains**; record the server's own timing fields (e.g. eval duration) in each record for latency reporting.
- A notebook (e.g. Colab) for running inference elsewhere, using the same modules rather than copied code.

28. Hardware diagnostics and documenting decisions

Diagnosing an unstable machine (freezes, reboots, GPU errors): - Run a GPU-only stress test with every logger in the background, each run in its own timestamped log directory: - `nvidia-smi dmon` at 1 Hz (power, temperature, clocks, utilisation); - live kernel messages (`dmesg -w` / `journalctl -kf`) for GPU Xid errors, PCIe AER, machine-check errors; - `sensors` every few seconds for CPU/board temperatures; - the test's own heartbeats. - A background loop calls `sync` every ~10 s so a hard freeze loses seconds of logs, not the page cache. - Write all PIDs to `pids.txt` ; `run_test.sh stop` kills exactly those. - After a reboot, read the last lines of each log to see what happened just before the freeze. A driver/library version mismatch after an update (`Failed to initialize NVML`) needs a reboot before GPU jobs start; the boot hook's GPU wait (§2) covers this.

Documenting decisions in code: - Docstrings and comments explain **why**, including the real incident that caused a design (e.g. "one corrupt image crashed a multi-hour eval, so failures are now recorded per item"). Future readers then don't remove the protection. - Record **verification dates** for external facts ("URL verified 2026-08-08", "train split has zero empty annotations — checked"). - For prompts, templates or constants reproduced from a paper, state in the code which parts are verbatim and which were reconstructed, and why. - Superseded scripts move to `scripts/legacy/` (kept for traceability of old results, not used by the pipeline) instead of being deleted or left mixed with current code. - `docs/` holds `pipeline.md` (design), `data.md` (datasets, splits, mappings), `reproducing.md` (commands, times) and `results.md` (numbers); the README links them.

29. Low-end GPUs and out-of-memory (OOM) playbook

For consumer and older cards (4–16 GB: GTX 10xx, RTX 20xx/30xx/40xx, T4, laptop GPUs). The goal: every job fits with headroom, an OOM never loses work, and the fixes applied are recorded in the config.

29.1 KNOW YOUR CARD BEFORE WRITING CODE

Record in `.env` / the config: GPU name, VRAM, compute capability, and whether it drives a display.

Capability	Check	Consequence
bf16 support	`torch.cuda.is_bf16_supported()`	Ampere (RTX 30xx) and newer: use bf16. Turing (RTX 20xx, T4) and Pascal (GTX 10xx): use fp16 with a gradient scaler; Pascal fp16 is slow, so consider fp32 for small models
FlashAttention-2	compute capability ≥ 8.0	otherwise use `attn_implementation="sdpa"` (works everywhere)
TF32	Ampere+	`torch.backends.cuda.matmul.allow_tf32 = True` for faster fp32 matmuls

Capability	Check	Consequence
Display attached	`nvidia-smi` shows Xorg / gnome-shell	the desktop takes 0.3–1 GB; run headless or budget for it

29.2 ESTIMATE MEMORY BEFORE RUNNING

Rules of thumb per parameter (verify by measuring, §29.3):

What	Bytes per parameter
Weights fp32 / bf16-fp16 / 8-bit / 4-bit (NF4)	4 / 2 / ~1 / ~0.5–0.6
Full fine-tuning with Adam, mixed precision (weights + grads + 2 optimizer states + master copy)	~16
LoRA / QLoRA	frozen weights at their storage size + a few % for adapters and their optimizer states

Plus **activations** (grow with batch × sequence length × layers; images become hundreds to thousands of tokens each in VLMs) and, for generation or cached scoring, the **KV cache**: `2 × layers × kv_heads × head_dim × bytes × tokens` per sequence. Write the estimate into the Setup notes; if it's above ~85% of free VRAM, choose a lever from §29.4 before the first run, not after the first OOM.

29.3 MEASURE, DON'T GUESS

- **Peak-memory preflight**: before a full run, run one forward + backward (training) or one forward (eval) on the **largest input in the dataset** (most frames, most image tiles, longest text), with the exact config. Print and save `torch.cuda.max_memory_allocated()` and `max_memory_reserved()`. The run is safe only if this peak leaves headroom (≥ 10–15%).
- Log peak memory per logging step to TensorBoard (`sys/gpu_mem_gb`); a slow upward trend means something is accumulating (see 29.6).
- For a real OOM, record a memory snapshot and look at what holds memory: `python torch.cuda.memory._record_memory_history(max_entries=100_000) ... # run until OOM / a few steps torch.cuda.memory._dump_snapshot("logs/mem_snapshot.pickle") # open at pytorch.org/memory_viz` `torch.cuda.memory_summary()` gives a text version.
- A **free-VRAM gate** at startup: print free/total memory and name the processes already on the GPU (`nvidia-smi --query-compute-apps=pid,process_name,used_memory --format=csv`); refuse to start (or warn loudly) if free memory is below the measured peak.

29.4 LEVERS, IN THE ORDER TO TRY THEM

Apply the cheapest lever that fits; record every lever used in the config, since several change results (resolution, frames, quantisation).

Training 1. **Batch size 1 + gradient accumulation** for the effective batch size you want (results nearly identical; only slower). 2. **Gradient checkpointing** (`gradient_checkpointing_kwargs={"use_reentrant": False}`): large activation savings for ~20–30% more compute. 3. **Mixed precision** (bf16, or fp16 + scaler on older cards). 4. **Parameter-efficient training**: LoRA instead of full fine-tuning; freeze the vision encoder; lower LoRA rank or fewer target modules if still too large. 5.

Quantise the frozen base: 4-bit NF4 with double quantisation (QLoRA), `prepare_model_for_kbit_training`. 6. **Memory-light optimiser**: `paged_adamw_8bit` (pages optimiser state to CPU on spikes), 8-bit Adam, or Adafactor. 7. **Shorter inputs**: cap sequence length, fewer image tiles (processor kwargs such as max patches / image size), lower resolution, fewer frames per clip. These **change the experiment**: apply identically to every compared system or report as a confound. 8. **Smaller model** from the same family. 9. **CPU offload** (`device_map="auto"` with `max_memory`, DeepSpeed ZeRO offload): fits almost anything, but much slower; last resort. - Trainer helpers: `auto_find_batch_size=True` (halves the batch on OOM), `eval_accumulation_steps` (moves eval predictions to CPU in chunks), `prediction_loss_only=True` when only loss is needed, and `remove_unused_columns=False` for vision inputs. - Group samples of similar size into batches (`group_by_length`) when batch size > 1, so one long sample doesn't set the memory peak for a whole batch.

Inference / evaluation 1. `torch.inference_mode()`; model in `eval()`. 2. Batch size 1 for variable-size inputs; bounded `max_new_tokens`. 3. Compute only the logits you need (`logits_to_keep=1`); vocabulary-sized logits over long image prompts are large. 4. Reuse the KV cache of a shared prompt instead of re-encoding it; pack short continuations into one forward with a custom attention mask (verify against the simple path, §10). 5. Quantised weights (4/8-bit) for evaluation too, matching training. 6. Move results to CPU immediately (`.item()`, `.cpu()`), never keep tensors from past items.

29.5 HANDLE OOM AT RUNTIME WITHOUT LOSING WORK

- Set `PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True` **before importing torch** (in code via `os.environ.setdefault` at the top of the entry script, or in `.env`); it reduces fragmentation OOMs on long runs with variable-size inputs.
- **Evaluation**: catch `torch.cuda.OutOfMemoryError` per item, delete references, `torch.cuda.empty_cache()`, optionally retry once with a reduced setting (e.g. fewer tiles) and **mark the record** with what was changed; otherwise record the item as failed and continue (§3). Report the failed count; a few failures are a footnote, many mean the config is wrong.
- **Training**: an OOM kills the step; rely on frequent checkpoints and auto-resume (§3, §22). After two OOMs at the same step, the runner stops retrying and reports, instead of looping — usually one oversized sample; find it from the step's data indices and cap or skip it explicitly (logged).
- `accelerate.utils.find_executable_batch_size` wraps a function and retries with a halved batch size on OOM; use it for throughput-oriented eval code.
- **Never fall back silently to CPU** or to a smaller model; it changes speed or results without notice. Fail with a message instead.

29.6 MEMORY LEAKS AND CREEPING USAGE

Usage that grows over steps means something is kept: - accumulating tensors that still hold the graph (`total_loss += loss` instead of `loss.item()`); - storing outputs/logits for all items on GPU; - Python references to old batches (in lists, closures, exception tracebacks — `except Exception as e` keeps frames alive until `e` is cleared); - growing caches (a KV cache not cropped/reset between items); - evaluation callbacks that don't switch back to `model.train()` or keep their outputs. Fix by keeping only Python scalars, deleting large objects, and running heavy phases (evaluation of a different model) in a **separate process**.

29.7 SHARE THE CARD CAREFULLY

- One GPU job at a time (lock file, §3). Two jobs that each fit alone will OOM together.
- Other things holding VRAM: inference servers keeping models loaded (e.g. Ollama with `keep_alive` — `ollama ps`, `ollama stop <model>`), image generation UIs, Jupyter kernels, browsers with hardware acceleration, the desktop. Check before every run (free-VRAM gate, 29.3).
- Run small-model baselines through a CPU or separate backend while training uses the GPU, when their memory can't collide.
- Release memory between models by ending the process, not by `del model` (allocator caches and CUDA context stay).

29.8 CPU RAM, DISK AND DATA LOADING ON SMALL MACHINES

- Load models with `low_cpu_mem_usage=True` / `device_map` and safetensors (memory-mapped) so a model isn't held twice in RAM while moving to GPU.
- DataLoader: `num_workers` 2–4 on small machines (each worker copies the dataset object); `pin_memory=True` only if RAM allows; `persistent_workers` to avoid re-spawning.
- Cache preprocessed inputs (resized images, extracted features) on the big disk when preprocessing is the bottleneck.
- Configure swap (on the big disk if needed) so a RAM spike slows the job instead of killing it; watch `dmesg` for the kernel OOM killer (`Out of memory: Killed process`), which looks like a silent exit.

29.9 THERMALS, POWER AND STABILITY

- Consumer and laptop GPUs throttle under multi-hour loads: monitor temperature and clocks (dashboard, §4; `nvidia-smi dmon`).
- Lowering the power limit (`sudo nvidia-smi -pl <watts>`) often costs little speed and reduces heat, freezes and PSU-related crashes.
- Ensure airflow; for laptops, keep on AC power and in a performance profile.
- Unstable machine → diagnose with §28 before blaming the code.

29.10 THROUGHPUT ON SLOW CARDS

- Measure where time goes (data loading, preprocessing, model forward) before optimising; often the vision encoder over many image tiles dominates.
- `torch.backends.cudnn.benchmark = True` when input sizes are fixed.
- Pre-tokenise and cache; avoid per-step Python work in the data path.
- `torch.compile` can help but adds compile time and memory; test it on the smoke run before using it on a long job.
- Estimate total runtime from the smoke run (s/item × items) and put it in the queue registry so ETAs and priorities are realistic.

29.11 OOM CHECKLIST

- [ ] Card capabilities recorded (VRAM, bf16, display attached)
- [ ] Memory estimate written; preflight on the largest input passed with ≥ 10–15% headroom
- [ ] `expandable_segments` set before torch import
- [ ] Free-VRAM gate at startup; nothing else holding the GPU
- [ ] Levers used recorded in the config, and applied identically to compared systems
- [ ] Per-item OOM handling in eval; frequent checkpoints + auto-resume in training

- [ ] Peak memory logged to TensorBoard; no upward creep over the smoke run
- [ ] No silent CPU / model fallback

Part 2 — Prompt for an AI coding agent

Paste this before giving the agent an experiment task:

```
Follow the engineering rules in trainingguide.md for all code you write in this
project. In particular:

1. Modularity: put new functionality in new modules that reuse existing
   loaders, metrics and result formats. Add new models/datasets/experiments as
   registry or YAML entries, not as copied scripts. Import heavy libraries
   inside functions.
2. Configuration: every run is defined by a YAML file under configs/
   (with inheritance from a base file). Save the resolved config, git commit,
   library versions and data hashes into each output directory. Validate
   configs before loading models.
3. Data location: all paths come from config.py / environment variables; data,
   checkpoints and caches live on the large disk (<DATA_ROOT>). Check the data
   is reachable before starting, and stop with a clear error if not.
4. Resumability: training checkpoints and auto-resumes; evaluation appends one
   record per item to a partial file, skips finished items on restart and
   writes the final file atomically; finished outputs are skipped; failed
   items are recorded, not fatal. Multiple experiments run through a queue
   runner with one subprocess per experiment, retries, a lock file, and
   --status / --list / --only flags.
5. Cron: add an @reboot hook script that waits for the data disk and GPU,
   then starts the dashboard and the runner; it must be idempotent. Launch
   long jobs with setsid nohup and unbuffered output. Kill processes by PID,
   never pkill -f.
6. Dashboard: provide a read-only stdlib HTTP dashboard (auto-refresh,
   /api/status JSON) with runner state, GPU state, per-experiment progress,
   rate, ETA, running metrics, failed counts, stall detection and log tail.
7. Before any full run: smoke-test every configuration on ~20 items per model
   family, check a known checkpoint reproduces its result, verify any
   optimisation against the simple implementation, and test kill-and-resume.
   Report these results before launching.
8. Log with timestamps, one log per experiment; seed everything from config;
   never overwrite finished results or checkpoints.
9. TensorBoard: every training run logs scalars, per-class validation
   metrics, confusion matrix, PR curves, a fixed set of prediction images,
   hparams and the config to runs/<run_name>/, and resumes into the same
   directory at the right global step. Start TensorBoard from the boot hook.
10. Result saving: save per-item records with scores for every class, raw
    output, grouping keys and a metadata block (config, checkpoint, data hash,
    git commit, versions). Metrics are recomputed from records by one shared
    function; headline numbers carry bootstrap intervals.
11. Visualizations: every figure is produced by a script from saved results,
    writes its plotted numbers to CSV, and uses one shared style module with
    fixed colours per system and colour-blind-safe palettes. Produce the
    standard set (curves, confusion matrix, per-class bars, PR/operating
    curves, calibration, embeddings, attribution maps, failure gallery).
12. Dataset grids and animations: seeded, code-generated class × sample
    grids, per-domain and per-camera grids, sequence strips, qualitative grids
```

```
   with GT/prediction and correctness colours (selection rule stated), and
   MP4/GIF animations of samples, predictions and attention, each saved with
   the list of item IDs used.
13. Paper figures and tables: sized to the venue's column widths, 7–9 pt
   embedded fonts, vector PDF + PNG, no titles, (a)/(b) panel labels, and
   LaTeX booktabs tables generated from result files and \input into the
   paper.
14. Garbage collection: everything you install or start must be listable and
   removable by the project alone. Tag every cron entry with
   '# managed-by=<project> id=<id> expires=<date>' and manage it through
   ops/cronctl.py (idempotent install, prune expired/missing-script
   entries, never touch unmanaged lines, back up first). Queue boot hooks
   retire themselves when all work is done. Use flock locks, PID files with
   command verification before killing, trash-then-expire for results and
   checkpoints, a YAML retention policy, and an ops/gc.py with a dry-run
   'audit' installed as a weekly report-only cron job. Every cleanup is dry
   run by default, restricted to allowed roots and logged.
15. Environment: machine-specific paths, ports and cache locations live in a
   gitignored .env (with a committed .env.example) pointing at the large
   disk; config.py loads it without overriding real environment variables;
   shell and cron scripts source it explicitly; secrets live separately.
16. Hugging Face: pin model and dataset revisions in YAML, load through one
   function, use the model's chat template, share one HF cache across
   projects on the big disk, pre-download then run with HF_HUB_OFFLINE=1,
   build TrainingArguments from YAML with auto-resume, and save
   self-contained models (safetensors + processor + labels + model card +
   config + metrics). Publish reusable models to SHARED_MODELS_ROOT as
   immutable versions with index.yaml, checksums and a test load, and load
   them elsewhere with a pinned version.
17. Workflow: a self-documenting Makefile (make help) is the single entry
   point, reads paths from config.py and has check/smoke/all/clean-derived
   targets. Training validates on task metrics (seeded subset, NaN-safe) at
   every checkpoint and preserves the best checkpoint outside the rotation.
   Downloads are parallel, resumable and failure-tolerant; dataset samples are
   seeded, stratified and pinned in git; analysis runs from cached predictions
   without a GPU, with committed summaries as fallback inputs.
18. Distribution: package large artifacts with a release script
   (inference-ready, SHA256SUMS, release notes); use self-contained,
   resumable handoff bundles for other machines; keep export stages separate
   with a preflight; benchmark bundles never send labels to the device.
   Comments record why a design exists and when external facts were verified;
   superseded scripts go to scripts/legacy/.
19. Low-end GPUs and OOM: record the card's capabilities (VRAM, bf16, display),
   estimate memory, and run a peak-memory preflight on the largest input
   before any full run. Apply levers in order (batch 1 + grad accumulation,
   gradient checkpointing, mixed precision, LoRA/frozen encoder, 4-bit base,
   8-bit/paged optimiser, shorter inputs, smaller model, offload) and record
   them in the config. Set expandable_segments before importing torch, gate
   on free VRAM, handle OOM per item in eval, checkpoint often in training,
   log peak memory, avoid leaks, one GPU job at a time, never fall back
   silently to CPU.

If a rule conflicts with the request or information is missing (paths,
ports, disk), ask instead of guessing. When done, report what was built, what
was tested and how, and how to monitor, stop and resume it.
```

THE AUTHOR

About the Author



Atanu Shuvam Roy

Department of CSE
University of Liberal Arts Bangladesh
Dhaka, Bangladesh

atanusroy.com

SUPPORTED BY
Robotics and Automation Research Lab (RARL), UK
www.rarl-lab.com

This series grows out of hands-on work in research and AI-driven experimentation. Its purpose is practical: to give people who already work with AI tools a clear, honest set of habits for writing papers and running experiments that other people can trust.

MORE FROM THE AUTHOR

Research, projects and further guides: **atanusroy.com**
About the lab that supports this work: **www.rarl-lab.com**

ALSO IN THE SERIES

Volume 1, Write Like a Researcher. A field guide to AI-assisted scientific writing: from research questions and methods to figures, claims and the reviewer's checklist.

RARL
Robotics & Automation Research Lab.

RARL AI Practitioner Series
Volume 2 · First edition, 2026
www.rarl-lab.com